

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-a40b03fcb6172b9b2.jsonl`  
- SHA-256: `aa8fb864f9a2c4406e4270c67412950b8457bfe0944b50966b4eebdbfbfaa4ee`  
- Source modified: `2026-07-06T17:29:03+00:00`  
- Imported at: `2026-07-08T15:59:35+00:00`  
- project: `wf_133acdee-dab`  
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`
```

Transcript

1. user (2026-07-06T17:26:48.059Z)

You are a COMPLETENESS reviewer.

REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer

DIFF UNDER REVIEW (read this first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-

indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff

GOAL OF THE CHANGE (wf294 incident): the Cloud Run rally-worker reported a run as status="success" with 0 segments even when the WASB shuttle-trajectory detector TIMED OUT (1800s) and produced zero trajectory. A detector timeout/abort was indistinguishable from a legitimately 0-rally video. The fix must make "detector failed/timed out" surface as a failed status (NOT "success") in report.json AND the completion marker, while a genuine 0-rally match stays a success.

WHAT THE FIX DID (4 files, all in the diff):

1. backend/pipeline/segmenters/trajectory_hybrid.py ? process_video() now returns a backend.results.Failure (instead of a bare []) on TWO detector-failure paths: the env healthcheck-not-ready path, and the "run_predict produced no trajectory" abort path. A trajectory-produced-but-zero-windows case still returns bare [] (a legit 0-rally). Return annotation widened to Union[List[Dict[str,Any]], Failure]; kept the # type: ignore[override].
2. backend/worker/__main__.py ? added _serialize_and_push_outputs() helper (writes+pushes intervals.csv + report.json). _fail(rc) now writes+pushes a status="failed" report.json / intervals.csv (0 segments), best-effort, BEFORE the M6 done-marker. The success path uses the same helper. The segmenter-Failure branch already routed to _fail(3).
3. tests/test_tracknet_hybrid.py ? the two abort tests now assert isinstance(res, Failure); added a test that zero-windows stays a bare [] (not a Failure).
4. tests/test_worker.py ? new test that a segmenter Failure -> rc 3 + report.json status="failed" + done-marker returncode 3/status="failed"; strengthened the 0-segment test to assert status="success".

KEY CONSUMER CODE TO CROSS-CHECK (read the real files, do not assume):

- backend/worker/__main__.py: cmd_infer (the full control flow: validation -> segmenter -> isinstance(result, Failure) -> _fail(3); the finally: emit_indexer_report; the success tail), _write_done_marker, _write_report_json, _write_intervals_csv.
- backend/orchestration.py: cached_process_result (~line 288, requires report status=="success" to hydrate), probe_process_cache (~line 255), _read_bucket_json, _resume_one_remote_job usage.
- backend/compute/cloud_run.py: CloudRunCompute.run_infer (reads the done-marker returncode + outputs/<id>/report.json).
- backend/api/job_worker.py: _resume_one_remote_job (waits for report.json to EXIST, then reads status; marks the job failed if status != success), _run_claimed_job.
- backend/results.py: Success/Failure/Result/unwrap_or_failure contract.
- backend/main.py (isinstance Failure branch ~1557), backend/pipeline/segmenters/gemini.py & motion.py (the existing Failure precedent), yolo_hybrid.py (a PARALLEL hybrid segmenter).

CONSTRAINTS THE FIX MUST RESPECT: ruff clean, mypy clean (trajectory_hybrid is NOT quarantined; worker.__main__ IS quarantined via mypy.ini ignore_errors), full suite green, coverage floor 70%.

The full suite is already green (1736 passed) and ruff/mypy clean ? so do NOT report "tests might fail" speculatively; report only concrete defects you can point at in the code.

Only report REAL defects: a wrong/edge behavior, a silent-failure path still leaking as success, a control-plane interaction regression, an idempotency/concurrency bug, a broken test assertion, or a genuine convention violation. Cite exact file:line. If a concern is speculative or you cannot ground it in the actual code, DO NOT report it.

YOUR LENS: Did the fix miss any OTHER path where a detector/pipeline failure still silently returns a bare [] (thus status="success"), inconsistent with the task's intent? Examine:

- trajectory_hybrid.process_video other bare-[] returns: invalid video_duration<=0, unknown sport (KeyError), missing API key, unknown provider, _resolve_runner NotImplementedError (detector_impl=native on a family with no native runner), and the Phase-3 case where the provider errors on EVERY window (all handoffs fail) -> empty ai_segments -> []. Which of these are genuinely "detector/pipeline FAILED" vs "ran, found nothing", and is leaving them as [] a real silent-failure gap given the incident?
- yolo_hybrid.py is a PARALLEL hybrid segmenter also dispatched by the worker. Does it have the SAME silent-failure shape (detector fail -> bare [])? The CODE_MAP twin-parity footgun: is the fix applied consistently, or does it create a wasb-vs-yolo/tracknet inconsistency?
- Is the fix's scope (2 paths) the right minimal scope, or does it leave a same-class hole that will reproduce the exact wf294 silent failure via a sibling path (e.g. healthcheck passes, run_predict returns a csv, but parse yields 0 points / the native runner wrote an empty file)?

Report concrete completeness gaps only (a specific path + why it's a failure-not-empty), file:line.

2. assistant (2026-07-06T17:26:53.977Z)

I'll start by reading the diff under review, then cross-check against the real consumer code and sibling segmenters.

3. user (2026-07-06T17:26:55.988Z)

```
1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..f96b495 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
28     -    ) -> List[Dict[str, Any]]:
29     -        # override-ignore: the ABC declares the Result contract (Success|Failure)
```

```

30 -         # but every live segmenter still returns a bare list ? the documented
31 -         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32 +     ) -> "Union[List[Dict[str, Any]], Failure]":
33 +         # Partial adoption of the Result contract (the ABC declares Success|Failure).
34 +         # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
35 +         # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
36 +         # genuine 0-rally video (which still returns a bare ``[]``). Every caller
37 +         # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
38 +         # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
39         label = self.display_label
40         # --- Sport profile + AI provider wiring (identical across segmenters) ---
41         sport_name = self.config.sport
42     @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
43         ok, msg = runner.healthcheck()
44         if not ok:
45             logger.error("%s detector environment not ready: %s", label, msg)
46         -         return []
47         +         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
48         +         # failed/retryable instead of writing an empty-but-"successful" result.
49         +         return Failure(
50             +             message=f"{label} detector environment not ready: {msg}",
51             +             is_recoverable=True,
52         +         )
53         logger.info("%s detector env OK: %s", label, msg)
54
55         # --- Phase 2: trajectory -> candidate rally windows ---
56     @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
57         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
58         if not csv_path:
59             logger.error("%s produced no trajectory; aborting.", label)
60         -         return []
61         +         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
62         +         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
63         +         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
64         +         # rather than a silent 0-segment "success" (the wf294 incident).
65         +         return Failure(
66             +             message=(
67                 +                 f"{label} detector produced no trajectory (timed out or aborted) ?
"
68                 +                 "detector failure, not a 0-rally video."
69             +             ),
70             +             is_recoverable=True,
71         +         )
72
73         points = runner.parse_trajectory_csv(csv_path)
74         logger.info(
75             diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76             index 461d7ca..88c50f6 100644
77             --- a/backend/worker/__main__.py
78             +++ b/backend/worker/__main__.py
79             @@ -114,6 +114,42 @@ def _write_report_json(
80                 )
81
82
83         +def _serialize_and_push_outputs(
84         +     scratch: str,
85         +     out_uri: str,

```

```

86     +   video_id: str,
87     +   segments: List[dict],
88     +   status: str,
89     +   storage_cfg: dict,
90     +   video_uri: str = "",
91     +) -> List[str]:
92     +   """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93     +   ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94     +
95     +   Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96     +   ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97     +   (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98     +   report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99     +   EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
100    +   out_local = os.path.join(scratch, "outputs", video_id)
101    +   os.makedirs(out_local, exist_ok=True)
102    +   _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103    +   _write_report_json(
104    +       video_id,
105    +       segments,
106    +       status,
107    +       os.path.join(out_local, "report.json"),
108    +       video_uri=video_uri,
109    +   )
110    +   out_prefix = out_uri.rstrip("/")
111    +   pushed: List[str] = []
112    +   for fn in sorted(os.listdir(out_local)):
113    +       uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114    +       storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115    +       pushed.append(uri)
116    +   return pushed
117    +
118    +
119    def _run_startup_checks(config: Any) -> bool:
120        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122    @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123    )
124
125    def _fail(rc: int) -> int:
126    -       # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127    -       # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128    -       # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129    +       # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130    +       # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131    +       # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132    +       # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133    +       # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134    +       # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135    +       # dispatcher's bucket-poll terminates FAST instead of hanging until its budget

```

```

expires.
136 +         try:
137 +             _serialize_and_push_outputs(
138 +                 scratch,
139 +                 args.out_uri,
140 +                 video_id,
141 +                 [],
142 +                 "failed",
143 +                 storage_cfg,
144 +                 str(getattr(args, "video_uri", "") or ""),
145 +             )
146 +         except Exception as e: # never mask the real failure on an artifact-push
hiccup
147 +             logger.warning("[worker] could not push failure artifacts: %s", e)
148 +             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149 +             if getattr(args, "done_uri", None):
150 +                 _write_done_marker(
151 +                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152 @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153             )
154
155             # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156 -             out_local = os.path.join(scratch, "outputs", video_id)
157 -             os.makedirs(out_local, exist_ok=True)
158 -             _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159 -             _write_report_json(
160 +             pushed = _serialize_and_push_outputs(
161 +                 scratch,
162 +                 args.out_uri,
163 +                 video_id,
164 +                 segments,
165 +                 status,
166 -                 os.path.join(out_local, "report.json"),
167 -                 video_uri=str(getattr(args, "video_uri", "") or ""),
168 +                 storage_cfg,
169 +                 str(getattr(args, "video_uri", "") or ""),
170             )
171
172 -             out_prefix = args.out_uri.rstrip("/")
173 -             pushed = []
174 -             for fn in sorted(os.listdir(out_local)):
175 -                 uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176 -                 storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177 -                 pushed.append(uri)
178 -
179             print(
180                 f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181             )
182             diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183             index 64909e7..c5ae442 100644
184             --- a/tests/test_tracknet_hybrid.py
185             +++ b/tests/test_tracknet_hybrid.py
186 @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187             from unittest.mock import MagicMock, patch
188
189             from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190 +from backend.results import Failure
191
192
193             def _window(start=1.0, end=4.0):
194 @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196             seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})

```

```

197         res = seg.process_video("v1", "clip.mp4")
198     -     assert res == []
199     +     # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200     +     # instead of writing an empty "success".
201     +     assert isinstance(res, Failure)
202     +     assert "environment not ready" in res.message
203     +     # Should bail before ever running inference.
204     +     mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207         mock_provider_registry.get.return_value = MagicMock()
208
209         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210     -     assert seg.process_video("v1", "clip.mp4") == []
211     +     res = seg.process_video("v1", "clip.mp4")
212     +     # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213     +     # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214     +     assert isinstance(res, Failure)
215     +     assert "no trajectory" in res.message and res.is_recoverable
216     +
217     +
218     +@_patched
219     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220     +     mock_runner_cls,
221     +     mock_probe,
222     +     mock_dur,
223     +     mock_extract,
224     +     mock_provider_registry,
225     +     mock_env,
226     +):
227     +     """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228     +     parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229     +     (0-rally "success"), NOT a Failure."""
230     +     mock_env.return_value = "dummy_key"
231     +     mock_dur.return_value = 30.0
232     +     mock_extract.return_value = True
233     +     mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
234     +     mock_provider_registry.get.return_value = MagicMock()
235     +
236     +     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237     +     res = seg.process_video("v1", "clip.mp4")
238     +     assert res == [] and not isinstance(res, Failure)
239     +     # The detector DID run (a genuine empty, not an abort).
240     +     mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243     @_patched
244     diff --git a/tests/test_worker.py b/tests/test_worker.py
245     index 96edbee..63cc18e 100644
246     --- a/tests/test_worker.py
247     +++ b/tests/test_worker.py
248     @@ -10,6 +10,7 @@ import json
249     import pytest
250
251     from backend.config import load_config
252     +from backend.results import Failure
253     from backend.worker import __main__ as wmain
254
255     # ----- pure helpers

```

```

256     @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
257
258
259     class _EmptySeg:
260     -     """A segmenter that finds nothing ? the cold-start failure mode."""
261     +     """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
262
263         def __init__(self, db, config):
264             pass
265     @@ -200,6 +201,20 @@ class _EmptySeg:
266         return []
267
268
269     +class _FailSeg:
270     +     """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271     +     trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
272     +
273     +     def __init__(self, db, config):
274     +         pass
275     +
276     +     def process_video(self, video_id, path, max_frames=None):
277     +         return Failure(
278     +             message="WASB detector produced no trajectory (timed out or aborted)",
279     +             is_recoverable=True,
280     +         )
281     +
282     +
283     def test_setup_logging_levels():
284         import logging
285
286     @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287         assert "0 segments" in msgs and "vidZ" in msgs
288         assert "detector_impl=native" in msgs # surfaces the resolved detector
289         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
290     -     assert (
291     -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292     -             "segment_count"
293     +     # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294     +     # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295     +     report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296     +     assert report["segment_count"] == 0 and report["status"] == "success"
297     +
298     +
299     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +     tmp_path, monkeypatch
301     +):
302     +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303     +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304     +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305     +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306     +     monkeypatch.setattr(
307     +         wmain.validator_registry,
308     +         "run_all_with_context",
309     +         lambda path, cfg: ([_OKResult()], {}),
310     +     )
311     +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312     +     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313     +     video = tmp_path / "in.mp4"

```

```

314 +     video.write_bytes(b"FAKE")
315 +     out = tmp_path / "bucket"
316 +     done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317 +
318 +     rc = wmain.main(
319 +         [
320 +             "infer",
321 +             "--video-uri",
322 +             str(video),
323 +             "--out-uri",
324 +             str(out),
325 +             "--scratch",
326 +             str(tmp_path / "s"),
327 +             "--segmenter",
328 +             "wasb_hybrid",
329 +             "--done-uri",
330 +             str(done),
331 +         ]
332 +         == 0
333 +     )
334 +     # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335 +     assert rc == 3
336 +     # report.json is written as an explicit FAILURE, not a false empty "success".
337 +     report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338 +     assert report["status"] == "failed" and report["segment_count"] == 0
339 +     assert report["segments"] == []
340 +     # ...and the completion marker the cloud dispatcher bucket-polls carries the same
    verdict.
341 +     marker = json.loads(done.read_text())
342 +     assert marker["returncode"] == 3 and marker["status"] == "failed"
343 +     assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347

```

4. user (2026-07-06T17:26:56.954Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20
21    import concurrent.futures
22    import logging
23    import os
24    import time
25    from typing import Any, Dict, List, Optional, Tuple, Union
26
27    import cv2

```



```

28
29 from backend.config.models import IndexingConfig
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.segmenters.base import BasePlaySegmenter
32 from backend.providers import provider_registry
33
34 # Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
35 from backend.results import Failure
36 from backend.sports import sport_registry
37 from backend.utils.paths import output_base
38 from backend.utils.video import extract_video_chunk, get_video_duration
39
40 logger = logging.getLogger(__name__)
41
42
43 def _fmt_ts(seconds: float) -> str:
44     seconds = max(0, int(seconds))
45     h, rem = divmod(seconds, 3600)
46     m, s = divmod(rem, 60)
47     return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50 class TrajectoryHybridSegmenter(BasePlaySegmenter):
51     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
52 boundaries."""
53
54     #: config section under `indexing` (velocity_threshold lives there), the
55     #: output subdir for trajectories, and the metadata detector-tag prefix.
56     family: str = ""
57     #: human-readable label used in log lines.
58     display_label: str = ""
59
60     def _build_runner(
61         self, idx_cfg: IndexingConfig
62     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
63         """Return (runner, detector-specific detection_params extras) for the default
64 (WSL) path."""
65         raise NotImplementedError
66
67     def _build_native_runner(
68         self, idx_cfg: IndexingConfig
69     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
70         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
71 with a
72 native runner override this; the base refuses with a clear message (M3.5: WASB
73 only)."""
74         raise NotImplementedError(
75             f"detector_impl='native' is not supported for the "
76             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
77 has a "
78             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
79 )
80
81     def _resolve_runner(
82         self, idx_cfg: IndexingConfig
83     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
84         """Resolve the detector runner, honouring the CPU-stub and Linux-native
85 overrides.
86
87         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
88         ``indexing.detector_impl``:
89         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
90 pipeline
91         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
92 swaps

```

```

84         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85         ``_build_native_runner``. Anything else (``"auto"``) delegates to
86         ``_build_runner``
87         (the WSL runner) so the default production path is unchanged
88         (``docs/DECOUPLED_COMPUTE/``).
89         """
90         # Safety net: accept raw dicts from legacy / test callers.
91         if isinstance(idx_cfg, dict):
92             idx_cfg = IndexingConfig(**idx_cfg)
93         impl = (
94             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
95         ).lower()
96         if impl == "stub":
97             from backend.pipeline.detectors.stub_runner import (
98                 StubConfig,
99                 StubDetectorRunner,
100             )
101             logger.info(
102                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103                 self.display_label or "trajectory",
104             )
105             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
106                 "detector_impl": "stub"
107             }
108         if impl in ("native", "native_wasb", "wasb_native"):
109             logger.info(
110                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
111                 self.display_label or "trajectory",
112             )
113             return self._build_native_runner(idx_cfg)
114         return self._build_runner(idx_cfg)
115
116     @staticmethod
117     def _probe_fps_width(video_path: str) -> tuple:
118         """Return (fps, frame_width) for a video, with sensible fallbacks."""
119         cap = cv2.VideoCapture(video_path)
120         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122         cap.release()
123         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
124
125     @staticmethod
126     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127         """Provider-reported exchange count, else a duration-based estimate.
128
129         One canonical implementation ? the twins had drifted (wasb relied on
130         ``int(None)`` raising into the fallback; same result, by accident).
131         """
132         shot_count = segment.get("shuttle_exchange_count")
133         if shot_count is None:
134             return max(3, int(duration / 0.8))
135         try:
136             return int(shot_count)
137         except (ValueError, TypeError):
138             return max(3, int(duration / 0.8))
139
140     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
141     -----
142     def _enrich_candidate_stats(
143         self,
144         cw: Dict[str, Any],
145         points: List[Any],
146         fps: float,
147         frame_width: float,

```

```

147         video_path: str,
148         idx_cfg: "IndexingConfig",
149     ) -> Dict[str, Any]:
150         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
151         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
152         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
153         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
154         return {
155             "peak_velocity": cw["peak_velocity"],
156             "mean_velocity": cw["mean_velocity"],
157             "active_frame_count": cw["active_frame_count"],
158             "track_id_count": cw["track_id_count"],
159         }
160
161     def _select_for_handoff(
162         self,
163         windows: List[Dict[str, Any]],
164         window_stats: List[Dict[str, Any]],
165         idx_cfg: "IndexingConfig",
166     ) -> List[int]:
167         """Indices of the candidate windows that proceed to the AI handoff (and thus may
168         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
170         Persisted candidates are NOT affected ? they remain the full superset for
offline
171         re-scoring."""
172         return list(range(len(windows)))
173
174     def process_video( # type: ignore[override]
175         self,
176         video_id: str,
177         video_path: str,
178         player_pool: Optional[List[str]] = None,
179         max_frames: Optional[int] = None,
180     ) -> "Union[List[Dict[str, Any]], Failure]":
181         # Partial adoption of the Result contract (the ABC declares Success|Failure).
182         # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
183         # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused with
a
184         # genuine 0-rally video (which still returns a bare ``[]``). Every caller
185         # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
186         # the bare-list success/empty paths are the documented incremental gap (CODE_MAP
gotchas).
187         label = self.display_label
188         # --- Sport profile + AI provider wiring (identical across segmenters) ---
189         sport_name = self.config.sport
190         try:
191             sport_profile = sport_registry.get(sport_name)
192         except KeyError as e:
193             logger.error(str(e))
194             return []
195
196         idx_cfg = self.config.indexing
197         skip_ai = bool(idx_cfg.skip_ai_handoff)
198         provider_name = self.config.ai_provider
199         if skip_ai:
200             model_name = "local-noai"
201             logger.info(
202                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
203                 "be emitted as rallies; no %s handoff, no API key needed.",

```

```

204         label,
205         provider_name,
206     )
207 else:
208     model_name = self.config.ai_model
209     api_key_env_var = f"{provider_name.upper()}_API_KEY"
210     api_key = os.environ.get(api_key_env_var)
211     if not api_key:
212         from backend.pipeline.segmenters._keyhint import api_key_hint
213
214         logger.error(
215             f"{api_key_env_var} environment variable is not set! "
216             f"To run the {provider_name} handoff, set your API key. "
217             + api_key_hint(api_key_env_var)
218         )
219         return []
220     try:
221         provider = provider_registry.get(
222             provider_name, api_key=api_key, model_name=model_name
223         )
224     except KeyError as e:
225         logger.error(str(e))
226         return []
227
228     video_duration = get_video_duration(video_path)
229     if video_duration <= 0:
230         logger.error("Invalid video duration.")
231         return []
232     fps, frame_width = self._probe_fps_width(video_path)
233
234     # --- Runner config + windowing thresholds ---
235     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
236     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
237     # it delegates to the subclass _build_runner (the real WSL/native runner).
238     try:
239         runner, runner_params = self._resolve_runner(idx_cfg)
240     except NotImplementedError as e:
241         # e.g. detector_impl="native" for a family without a native runner
242         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
243         # crashing.
244         logger.error("%s: %s", label, e)
245         return []
246
247     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
248     merge_gap = idx_cfg.dead_time_merge_gap
249     min_window_duration = idx_cfg.min_action_window_duration
250     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
251     IndexingConfig.
252     max_window_duration = idx_cfg.max_window_duration
253     window_min_split_gap = idx_cfg.window_min_split_gap
254     window_hard_cap = idx_cfg.window_hard_cap
255     window_inactivity_end = idx_cfg.window_inactivity_end
256     inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
257     inactivity_min_density = idx_cfg.inactivity_min_density
258     inactivity_temporal_density = idx_cfg.inactivity_temporal_density
259     window_blob_fallback = idx_cfg.window_blob_fallback
260     window_blob_factor = idx_cfg.window_blob_factor
261     handoff_padding = idx_cfg.ai_handoff_padding
262     ai_max_workers = idx_cfg.ai_max_workers
263     log_candidates = idx_cfg.log_yolo_candidates
264     store_candidates = idx_cfg.store_yolo_candidates
265
266     detection_params = {
267         "detector": self.name,

```

```

266         "velocity_thresh": velocity_thresh,
267         "merge_gap": merge_gap,
268         "min_action_window_duration": min_window_duration,
269         **runner_params,
270     }
271
272     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
273     ok, msg = runner.healthcheck()
274     if not ok:
275         logger.error("%s detector environment not ready: %s", label, msg)
276         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
277         # failed/retryable instead of writing an empty-but-"successful" result.
278         return Failure(
279             message=f"{label} detector environment not ready: {msg}",
280             is_recoverable=True,
281         )
282     logger.info("%s detector env OK: %s", label, msg)
283
284     # --- Phase 2: trajectory -> candidate rally windows ---
285     # Trajectory detectors process the whole video in one pass (they carry
286     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
287     t_start = time.time()
288     out_dir = os.path.join(output_base(self.config), self.family)
289     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
290     if not csv_path:
291         logger.error("%s produced no trajectory; aborting.", label)
292         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
293         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
294         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
295         # rather than a silent 0-segment "success" (the wf294 incident).
296         return Failure(
297             message=(
298                 f"{label} detector produced no trajectory (timed out or aborted) ? "
299                 "detector failure, not a 0-rally video."
300             ),
301             is_recoverable=True,
302         )
303
304     points = runner.parse_trajectory_csv(csv_path)
305     logger.info(
306         "Parsed %d trajectory points (%d visible).",
307         len(points),
308         sum(1 for p in points if p.visible),
309     )
310
311     all_candidate_windows = runner.trajectory_to_action_windows(
312         points,
313         fps=fps,
314         frame_width=frame_width,
315         chunk_start=0.0,
316         velocity_thresh=velocity_thresh,
317         merge_gap=merge_gap,
318         min_window_duration=min_window_duration,
319         chunk_index=0,
320         max_window_duration=max_window_duration,
321         min_split_gap=window_min_split_gap,
322         window_hard_cap=window_hard_cap,
323         window_inactivity_end=window_inactivity_end,
324         inactivity_rally_thresh=inactivity_rally_thresh,
325         inactivity_min_density=inactivity_min_density,
326         inactivity_temporal_density=inactivity_temporal_density,

```

```

327         window_blob_fallback=window_blob_fallback,
328         window_blob_factor=window_blob_factor,
329     )
330     logger.info(
331         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
332         label,
333         time.time() - t_start,
334         len(all_candidate_windows),
335     )
336
337     if log_candidates:
338         for cw in all_candidate_windows:
339             logger.info(
340                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
341                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
342                 f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
343             )
344
345     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
346     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
347     # persistence and the handoff gate below.
348     window_stats = [
349         self._enrich_candidate_stats(
350             cw, points, fps, frame_width, video_path, idx_cfg
351         )
352         for cw in all_candidate_windows
353     ]
354
355     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
356     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
357     if store_candidates:
358         for i, cw in enumerate(all_candidate_windows):
359             candidate_ids[i] = self.db.add_candidate_segment(
360                 video_id,
361                 detector=self.name,
362                 start_time=cw["start"],
363                 end_time=cw["end"],
364                 chunk_index=cw["chunk_index"],
365                 confidence=min(1.0, cw["peak_velocity"]),
366                 stats=window_stats[i],
367                 detection_params=detection_params,
368             )
369
370     if not all_candidate_windows:
371         return []
372
373     # --- Phase 3: AI provider handoff over padded candidate clips ---
374     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
375     # candidates above stay the full superset ? only the served play_segments are
gated.
376     handoff_indices = self._select_for_handoff(
377         all_candidate_windows, window_stats, idx_cfg
378     )
379
380     ai_segments: List[dict] = []
381     if skip_ai:
382         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
383         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
384         # falls back to the duration-based estimate (no AI exchange count).

```

```

385     ai_segments = [
386         {
387             "start_time": all_candidate_windows[wi]["start"],
388             "end_time": all_candidate_windows[wi]["end"],
389             "shuttle_exchange_count": None,
390             "shot_count_confidence": "LocalNoAI",
391             "_candidate_id": candidate_ids[wi],
392         }
393         for wi in handoff_indices
394     ]
395     logger.info(
396         "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
397         "
398         "(no AI handoff).",
399         len(ai_segments),
400     )
401 else:
402     handoff_dir = os.path.join(
403         output_base(self.config), f"chunks_{provider_name}_handoff"
404     )
405     os.makedirs(handoff_dir, exist_ok=True)
406     logger.info(
407         "Phase 3: %s validation on %d candidate windows...",
408         provider_name,
409         len(handoff_indices),
410     )
411     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
412         w_start = max(0, window["start"] - handoff_padding)
413         w_end = min(video_duration, window["end"] + handoff_padding)
414         w_dur = w_end - w_start
415         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
416         if not extract_video_chunk(
417             video_path, w_start, w_end, w_path, reencode=True
418         ):
419             return []
420         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
421         segments, metrics = provider.analyze_video(
422             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
423         )
424         # B2: a provider failure returns [] with metrics["error"] set ? that is
425         # "we don't know", NOT "no rallies in this window". Surface it loudly so
426         # partial run isn't mistaken for a complete one (mirrors the gemini
427         # segmenter).
428         if metrics.get("error"):
429             logger.warning(
430                 "[Handoff %d] provider error ? window %d skipped: %s",
431                 wi + 1,
432                 wi,
433                 metrics["error"],
434             )
435         valid = []
436         for seg in segments:
437             try:
438                 seg["start_time"] = float(seg["start_time"]) + w_start
439                 seg["end_time"] = float(seg["end_time"]) + w_start
440                 seg["_candidate_id"] = candidate_ids[wi]
441                 valid.append(seg)
442             except (ValueError, TypeError, KeyError) as e:
443                 logger.warning(
444                     "Dropping segment with unparseable timestamps: %s ? %s",
445                     seg,
446                     e,
447                 )

```

```

447         try:
448             os.remove(w_path)
449         except OSError:
450             pass
451         return valid
452
453     with concurrent.futures.ThreadPoolExecutor(
454         max_workers=ai_max_workers
455     ) as executor:
456         futures = [
457             executor.submit(
458                 process_candidate_window, wi, all_candidate_windows[wi]
459             )
460             for wi in handoff_indices
461         ]
462         for future in futures:
463             ai_segments.extend(future.result())
464     try:
465         os.rmdir(handoff_dir)
466     except OSError:
467         pass
468     logger.info(
469         "Phase 3 completed. AI confirmed %d semantic play segments.",
470         len(ai_segments),
471     )
472
473     # --- Phase 4: DB population + candidate linking ---
474     segments_data = []
475     for segment in ai_segments:
476         try:
477             start = float(segment["start_time"])
478             end = float(segment["end_time"])
479             dur = end - start
480             metadata = {
481                 "shot_count": self._shot_count_from(segment, dur),
482                 "shot_count_confidence": segment.get(
483                     "shot_count_confidence", "Unknown"
484                 ),
485                 "detector": f"{self.family}-hybrid-{model_name}",
486             }
487             segment_id = self.db.add_play_segment(
488                 video_id, start, end, None, None, metadata
489             )
490             if segment_id:
491                 candidate_id = segment.get("_candidate_id")
492                 if candidate_id is not None:
493                     self.db.link_candidate_to_segment(candidate_id, segment_id)
494                 segments_data.append(
495                     {
496                         "id": segment_id,
497                         "start_time": start,
498                         "end_time": end,
499                         "duration": dur,
500                         "metadata": metadata,
501                     }
502                 )
503         except Exception as e:
504             logger.warning("Skipping invalid segment struct: %s", e)
505
506     return segments_data
507

```

5. assistant (2026-07-06T17:27:03.230Z)

I have the diff and the segmenter. Now let me cross-check the sibling segmenter (yolo_hybrid),

the worker control flow, and the consumer code paths.

6. user (2026-07-06T17:27:04.699Z)

```
1      """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2      (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4      Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5      (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6      can be consumed by the future fusion segmenter; this segmenter just orchestrates
7      chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9      Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11     detection uses torchvision's permissively-licensed (BSD) detectors and the
12     association/tracking that YOLO.track() used to bundle in is provided by ByteTrack
13     via the standalone Apache-2.0 ``trackers`` package (D13/P12 migration, #386 ?
14     previously ``supervision.ByteTrack``, removed upstream in supervision 0.30.0; see
15     ``person_detector.make_tracker`` for the behaviour-pinning details).
16
17     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
18     DB `detector` tags, even though no YOLO is involved anymore.
19     """
20
21     import concurrent.futures
22     import logging
23     import os
24     import time
25     from typing import Any, Dict, List, Tuple
26
27     from backend.pipeline.detectors.person_detector import PersonDetector
28     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
29     from backend.providers import provider_registry
30     from backend.sports import sport_registry
31     from backend.utils.paths import output_base
32     from backend.utils.video import extract_video_chunk, get_video_duration
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
39         seconds = max(0, int(seconds))
40         h, rem = divmod(seconds, 3600)
41         m, s = divmod(rem, 60)
42         return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45     class HybridYoloSegmenter(BasePlaySegmenter):
46         def __init__(self, db, config):
47             super().__init__(db, config)
48             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
49             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
50             self.person = PersonDetector(config)
51
52         @property
53         def name(self) -> str:
54             return "yolo_hybrid"
55
56         @property
57         def description(self) -> str:
58             return (
59                 "Two-stage pipeline: torchvision person detection + ByteTrack for fast "
60                 "candidate filtering, then an AI provider for high-precision semantic
```

```

boundaries."
61         )
62
63     def _resolve_provider(self, video_path: str):
64         """Phase 0: resolve sport profile + AI provider (with API key).
65
66         Returns the configured provider instance, or ``None`` to signal that
67         ``process_video`` should early-return ``[]`` (bad sport, missing API key,
68         or unknown provider). Behaviour is identical to the inlined version.
69         """
70         # Get sport profile config
71         sport_name = self.config.sport
72         try:
73             sport_registry.get(sport_name)
74         except KeyError as e:
75             logger.error(str(e))
76             return None
77
78         # Get AI provider and model config
79         provider_name = self.config.ai_provider
80         model_name = self.config.ai_model
81
82         # Get appropriate API key based on the provider
83         api_key_env_var = f"{provider_name.upper()}_API_KEY"
84         api_key = os.environ.get(api_key_env_var)
85         if not api_key:
86             from backend.pipeline.segmenters._keyhint import api_key_hint
87
88             logger.error(
89                 f"{api_key_env_var} environment variable is not set! "
90                 f"To run the {provider_name} segmenter, set your API key. "
91                 + api_key_hint(api_key_env_var)
92             )
93             return None
94
95         try:
96             provider = provider_registry.get(
97                 provider_name, api_key=api_key, model_name=model_name
98             )
99         except KeyError as e:
100             logger.error(str(e))
101             return None
102
103         logger.info(
104             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
105 {video_path}"
106         )
107         return provider
108
109     def _collect_candidate_windows(
110         self,
111         video_id: str,
112         video_path: str,
113         video_duration: float,
114         idx_cfg: Dict[str, Any],
115         chunk_duration: float,
116         chunk_overlap: float,
117         velocity_thresh: float,
118         merge_gap: float,
119         frame_skip: int,
120         chunks_dir: str,
121     ) -> List[Dict[str, Any]]:
122         """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
123         sequential), then dedup-merge overlapping windows across chunks. Verbatim
124         move."""

```

```

123         step = chunk_duration - chunk_overlap
124         chunk_starts = []
125         t = 0.0
126         while t < video_duration:
127             chunk_starts.append(t)
128             t += step
129
130         num_chunks = len(chunk_starts)
131         logger.info(
132             f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks."
133         )
134
135         all_candidate_windows = []
136
137         t_start = time.time()
138         prefetch_workers = idx_cfg.yolo_prefetch_workers
139
140         if num_chunks > 1 and prefetch_workers > 0:
141             # --- PIPELINED PROCESSING ---
142             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
143             # extraction is pure I/O. We pre-extract upcoming chunks in background
144             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
145             logger.info(
146                 f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)..."
147             )
148
149             extract_executor = concurrent.futures.ThreadPoolExecutor(
150                 max_workers=prefetch_workers
151             )
152
153             def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
154                 chunk_end = min(cs + chunk_duration, video_duration)
155                 actual_dur = chunk_end - cs
156                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
157                 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
158                 return (ci, cs, chunk_path, success)
159
160             # Submit all extractions upfront ? they'll queue and run as workers free up
161             extract_futures = [
162                 extract_executor.submit(_extract_chunk, ci, cs)
163                 for ci, cs in enumerate(chunk_starts)
164             ]
165
166             # Process results in order: wait for extraction, then run inference on GPU
167             for future in extract_futures:
168                 ci, chunk_start, chunk_path, success = future.result()
169                 chunk_end = min(chunk_start + chunk_duration, video_duration)
170                 logger.info(
171                     f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
172                 )
173
174                 if not success:
175                     logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
176                     continue
177
178                 windows = self.person.extract_action_windows_from_chunk(
179                     chunk_path,
180                     chunk_start,
181                     velocity_thresh,
182                     merge_gap,
183                     frame_skip=frame_skip,
184                     chunk_index=ci,

```

```

185         )
186         logger.info(
187             f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
188         )
189         all_candidate_windows.extend(windows)
190
191         try:
192             os.remove(chunk_path)
193         except Exception:
194             pass
195
196         extract_executor.shutdown(wait=False)
197     else:
198         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
199         for ci, chunk_start in enumerate(chunk_starts):
200             chunk_end = min(chunk_start + chunk_duration, video_duration)
201             actual_dur = chunk_end - chunk_start
202             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
203
204             logger.info(
205                 f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
206             )
207             success = extract_video_chunk(
208                 video_path, chunk_start, actual_dur, chunk_path
209             )
210             if not success:
211                 continue
212
213             windows = self.person.extract_action_windows_from_chunk(
214                 chunk_path,
215                 chunk_start,
216                 velocity_thresh,
217                 merge_gap,
218                 frame_skip=frame_skip,
219                 chunk_index=ci,
220             )
221             logger.info(
222                 f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
223             )
224             all_candidate_windows.extend(windows)
225
226             try:
227                 os.remove(chunk_path)
228             except Exception:
229                 pass
230
231     # Deduplicate overlapping candidate windows across chunks (chunks overlap by
232     # chunk_overlap, so the same rally can surface in two adjacent chunks).
233     def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
234         fa, fb = a["active_frame_count"], b["active_frame_count"]
235         total = fa + fb or 1
236         return {
237             "start": a["start"],
238             "end": max(a["end"], b["end"]),
239             "active_frame_count": fa + fb,
240             "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
241             # Frame-weighted mean so longer sub-windows dominate proportionally.
242             "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb)
243             / total,
244             "track_id_count": max(a["track_id_count"], b["track_id_count"]),
245             "chunk_index": a["chunk_index"],
246         }
247
248     if all_candidate_windows:

```

```

249         all_candidate_windows.sort(key=lambda w: w["start"])
250         merged_windows = [all_candidate_windows[0]]
251         for current in all_candidate_windows[1:]:
252             last = merged_windows[-1]
253             if current["start"] <= last["end"]: # Overlap
254                 merged_windows[-1] = _merge_stats(last, current)
255             else:
256                 merged_windows.append(current)
257         all_candidate_windows = merged_windows
258
259         logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
260         logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
261
262         return all_candidate_windows
263
264     def _persist_candidates(
265         self,
266         video_id: str,
267         all_candidate_windows: List[Dict[str, Any]],
268         detection_params: Dict[str, Any],
269         log_candidates: bool,
270         store_candidates: bool,
271     ) -> List[Any]:
272         """Phase 2b: per-rally console log + persist raw candidates for the fine-tuning
273         dataset. Returns window-index -> candidate_id (None where not stored). Verbatim
274         move."""
275         # Per-rally console log (final, full-video timestamps) for video cross-
276         verification.
277         if log_candidates:
278             for w in all_candidate_windows:
279                 logger.info(
280                     f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
281                     f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']} "
282                     f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
283                     f"tracks={w['track_id_count']}"
284                 )
285             # Persist raw candidates for the fine-tuning dataset. Map window index ->
286             candidate_id
287             # so confirmed AI segments can be linked back in Phase 4.
288             candidate_ids = [None] * len(all_candidate_windows)
289             if store_candidates:
290                 for i, w in enumerate(all_candidate_windows):
291                     stats = {
292                         "peak_velocity": w["peak_velocity"],
293                         "mean_velocity": w["mean_velocity"],
294                         "active_frame_count": w["active_frame_count"],
295                         "track_id_count": w["track_id_count"],
296                     }
297                     # Use peak velocity (capped at 1.0) as a coarse generic confidence
298                     score.
299                     confidence = min(1.0, w["peak_velocity"])
300                     candidate_ids[i] = self.db.add_candidate_segment(
301                         video_id,
302                         detector="yolo_hybrid",
303                         start_time=w["start"],
304                         end_time=w["end"],
305                         chunk_index=w["chunk_index"],
306                         confidence=confidence,
307                         stats=stats,
308                         detection_params=detection_params,
309                     )
310             return candidate_ids

```

```

309
310     def _run_ai_handoff(
311         self,
312         video_id: str,
313         video_path: str,
314         video_duration: float,
315         all_candidate_windows: List[Dict[str, Any]],
316         candidate_ids: List[Any],
317         provider,
318         sport_profile,
319         provider_name: str,
320         chunk_duration: float,
321         handoff_padding: float,
322         ai_max_workers: int,
323     ) -> List[dict]:
324         """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
325         the provider for semantic boundaries, in parallel. Verbatim move."""
326         gemini_segments = []
327         gemini_dir = os.path.join(
328             output_base(self.config), f"chunks_{provider_name}_handoff"
329         )
330         os.makedirs(gemini_dir, exist_ok=True)
331
332         logger.info(
333             f"Starting Phase 3: AI Provider ({provider_name}) Validation on
334             {len(all_candidate_windows)} candidate windows..."
335         )
336
337         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
338             w_start = max(0, window["start"] - handoff_padding)
339             w_end = min(video_duration, window["end"] + handoff_padding)
340             w_dur = w_end - w_start
341
342             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
343             # Re-encode short handoff clips to avoid keyframe-boundary drift
344             success = extract_video_chunk(
345                 video_path, w_start, w_dur, w_path, reencode=True
346             )
347             if not success:
348                 return []
349
350             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
351             segments, metrics = provider.analyze_video(
352                 w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
353             )
354             # B2: a provider failure returns [] with metrics["error"] set ? that is
355             # "we don't know", NOT "no rallies in this window". Surface it loudly so a
356             # partial run isn't mistaken for a complete one (mirrors the gemini
357             segmenter).
358             if metrics.get("error"):
359                 logger.warning(
360                     "[Handoff %d] provider error ? window %d skipped: %s",
361                     wi + 1,
362                     wi,
363                     metrics["error"],
364                 )
365
366             # Coordinate mapping ? shift local timestamps back to full-video time
367             valid_segments = []
368             for segment in segments:
369                 try:
370                     segment["start_time"] = float(segment["start_time"]) + w_start
371                     segment["end_time"] = float(segment["end_time"]) + w_start
372                     # Stamp the source candidate so Phase 4 can link
373                     prediction->confirmation.

```

```

371         segment["_candidate_id"] = candidate_ids[wi]
372         valid_segments.append(segment)
373     except (ValueError, TypeError, KeyError) as e:
374         logger.warning(
375             f"Dropping segment with unparseable timestamps: {segment} ? {e}"
376         )
377
378     try:
379         os.remove(w_path)
380     except OSError:
381         pass
382
383     return valid_segments
384
385 with concurrent.futures.ThreadPoolExecutor(
386     max_workers=ai_max_workers
387 ) as executor:
388     futures = [
389         executor.submit(process_candidate_window, wi, w)
390         for wi, w in enumerate(all_candidate_windows)
391     ]
392     # Iterate in submission order to preserve temporal ordering
393     for future in futures:
394         gemini_segments.extend(future.result())
395
396     try:
397         os.rmdir(gemini_dir)
398     except OSError:
399         pass
400
401     logger.info(
402         f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
403     )
404     return gemini_segments
405
406 def _populate_db(
407     self, video_id: str, gemini_segments: List[dict], model_name: str
408 ) -> List[Dict[str, Any]]:
409     """Phase 4: DB Population ? write confirmed AI segments, link them back to their
410     source candidate, and build the returned segments_data. Verbatim move."""
411     segments_data = []
412     for segment in gemini_segments:
413         try:
414             start = float(segment["start_time"])
415             end = float(segment["end_time"])
416             dur = end - start
417
418             shot_count = segment.get("shuttle_exchange_count")
419             if shot_count is None:
420                 shot_count = max(3, int(dur / 0.8))
421             else:
422                 try:
423                     shot_count = int(shot_count)
424                 except (ValueError, TypeError):
425                     shot_count = max(3, int(dur / 0.8))
426
427             metadata = {
428                 "shot_count": shot_count,
429                 "shot_count_confidence": segment.get(
430                     "shot_count_confidence", "Unknown"
431                 ),
432                 "detector": f"yolo-hybrid-{model_name}",
433             }
434

```

```

435         segment_id = self.db.add_play_segment(
436             video_id, start, end, None, None, metadata
437         )
438         if segment_id:
439             # Link back to the candidate that produced this segment. A candidate
440             # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
441             # the last one ? the link table records "this candidate was
confirmed".
442             candidate_id = segment.get("_candidate_id")
443             if candidate_id is not None:
444                 self.db.link_candidate_to_segment(candidate_id, segment_id)
445             segments_data.append(
446                 {
447                     "id": segment_id,
448                     "start_time": start,
449                     "end_time": end,
450                     "duration": dur,
451                     "metadata": metadata,
452                 }
453             )
454         except Exception as e:
455             logger.warning(f"Skipping invalid segment struct: {e}")
456
457     return segments_data
458
459 def process_video(
460     self,
461     video_id: str,
462     video_path: str,
463     player_pool: List[str] = None,
464     max_frames: int = None,
465 ) -> List[Dict[str, Any]]:
466     self.person.lazy_load_model()
467
468     # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
469     provider = self._resolve_provider(video_path)
470     if provider is None:
471         return []
472
473     # Re-read the config values needed downstream (pure deterministic reads).
474     idx_cfg = self.config.indexing
475     sport_profile = sport_registry.get(self.config.sport)
476     provider_name = self.config.ai_provider
477     model_name = self.config.ai_model
478
479     video_duration = get_video_duration(video_path)
480     if video_duration <= 0:
481         logger.error("Invalid video duration.")
482         return []
483
484     chunk_duration = idx_cfg.chunk_duration
485     chunk_overlap = idx_cfg.chunk_overlap
486     velocity_thresh = (
487         idx_cfg.yolo_velocity_threshold
488     ) # Fraction of frame width per second
489     merge_gap = idx_cfg.dead_time_merge_gap
490     frame_skip = idx_cfg.yolo_frame_skip
491     tracker_config = idx_cfg.yolo_tracker
492     handoff_padding = idx_cfg.ai_handoff_padding
493     ai_max_workers = idx_cfg.ai_max_workers
494     log_candidates = idx_cfg.log_yolo_candidates
495     store_candidates = idx_cfg.store_yolo_candidates
496

```



```

497     # Snapshot of the params that produced these detections ? stored alongside each
498     # candidate so a fine-tuning run can reproduce / correlate against them.
499     detection_params = {
500         "velocity_thresh": velocity_thresh,
501         "merge_gap": merge_gap,
502         "frame_skip": frame_skip,
503         "tracker": tracker_config,
504         "detector_model": idx_cfg.detector_model,
505         "detector_score_threshold": idx_cfg.detector_score_threshold,
506         "min_action_window_duration": idx_cfg.min_action_window_duration,
507     }
508
509     chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
510     os.makedirs(chunks_dir, exist_ok=True)
511
512     # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
513     all_candidate_windows = self._collect_candidate_windows(
514         video_id,
515         video_path,
516         video_duration,
517         idx_cfg,
518         chunk_duration,
519         chunk_overlap,
520         velocity_thresh,
521         merge_gap,
522         frame_skip,
523         chunks_dir,
524     )
525
526     # Phase 2b: log + persist raw candidates (window index -> candidate_id).
527     candidate_ids = self._persist_candidates(
528         video_id,
529         all_candidate_windows,
530         detection_params,
531         log_candidates,
532         store_candidates,
533     )
534
535     try:
536         os.rmdir(chunks_dir)
537     except Exception:
538         pass
539
540     # Phase 3: AI Provider Handoff
541     gemini_segments = self._run_ai_handoff(
542         video_id,
543         video_path,
544         video_duration,
545         all_candidate_windows,
546         candidate_ids,
547         provider,
548         sport_profile,
549         provider_name,
550         chunk_duration,
551         handoff_padding,
552         ai_max_workers,
553     )
554
555     # Phase 4: DB Population
556     return self._populate_db(video_id, gemini_segments, model_name)
557
558
559     # Register the segmenter
560     segmenter_registry.register(HybridYoloSegmenter)
561

```

7. user (2026-07-06T17:27:05.257Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20
21     from __future__ import annotations
22
23     import argparse
24     import csv
25     import json
26     import logging
27     import os
28     import re
29     import sys
30     import tempfile
31     from typing import Any, List
32
33     from backend import storage
34     from backend.compute import ComputeJobSpec, get_compute_backend
35     from backend.config import (
36         StartupCheckError,
37         enforce_startup_checks,
38         load_config,
39         probe_cuda_available,
40     )
41     from backend.infrastructure.database import Database
42     from backend.pipeline.segmenters.base import segmenter_registry
43     from backend.pipeline.validators.base import registry as validator_registry
44     from backend.reporting import emit_indexer_report
45     from backend.results import Failure
46     from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver
47     from backend.utils.hashing import compute_video_id
48
49     logger = logging.getLogger("backend.worker")
50
51     def _coerce(v: str) -> Any:
52         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
53         str."""
54         low = v.lower()
55         if low in ("true", "false"):
56             return low == "true"
57         for cast in (int, float):
58             try:
59                 return cast(v)
60             except ValueError:
61                 pass
62         return v
```

```

59
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71                 setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],

```

```

122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174 def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175     """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the

```

```

176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
180         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
181         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
182         from backend.infrastructure.execution_policy import PolicyTimeout
183
184         spec = ComputeJobSpec(
185             video_uri=args.video_uri,
186             out_uri=args.out_uri,
187             segmenter=args.segmenter,
188             config_overrides=tuple(args.set or ()),
189         )
190         try:
191             result = compute.run_infer(spec)
192         except PolicyTimeout as e:
193             logger.warning(
194                 "[worker] remote compute did not complete (no marker within budget): %s", e
195             )
196             return 4
197         logger.info(
198             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
199             result.video_id,
200             result.returncode,
201             result.outputs_uri,
202         )
203         return result.returncode
204
205
206     def _write_done_marker(
207         done_uri: str,
208         video_id: str,
209         returncode: int,
210         segment_count: int,
211         status: str,
212         storage_cfg: dict,
213         scratch: str,
214     ) -> None:
215         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
216         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
217         try:
218             marker = {
219                 "video_id": video_id,
220                 "returncode": returncode,
221                 "segment_count": segment_count,
222                 "status": status,
223             }
224             local = os.path.join(scratch, "_done.json")
225             with open(local, "w", encoding="utf-8") as f:
226                 json.dump(marker, f)
227             storage.put(local, done_uri, config=storage_cfg)
228             logger.info("[worker] wrote completion marker -> %s", done_uri)
229         except Exception as e: # never fail a good run because the marker push hiccuped
230             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
231
232
233     def cmd_infer(args: argparse.Namespace) -> int:

```

```

234     config = load_config(args.config) if args.config else load_config()
235     _apply_overrides(config, args.set)
236     if not _run_startup_checks(config):
237         return 2
238
239     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
240     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
241     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
242     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
243     compute = get_compute_backend(config)
244     if compute is not None:
245         return _dispatch_remote(compute, args)
246
247     storage_cfg = config.storage.model_dump()
248
249     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
250     os.makedirs(scratch, exist_ok=True)
251     config.output.output_dir = scratch
252
253     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
254     local_video = storage.fetch(
255         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
256     )
257     print(f"[worker] pulled {args.video_uri} -> {local_video}")
258
259     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
260     video_id = compute_video_id(local_video)
261
262     # 3. VALIDATE (same registry as the main CLI).
263     val_results, val_context = validator_registry.run_all_with_context(
264         local_video, config
265     )
266     failures = [r for r in val_results if not r.passed]
267     db = Database(os.path.join(scratch, config.output.db_name))
268     db.add_video(
269         video_id,
270         local_video,
271         "failed" if failures else "processed",
272         [r.model_dump() for r in val_results],
273     )
274
275     def _fail(rc: int) -> int:
276         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
277         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
278         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283         try:
284             _serialize_and_push_outputs(
285                 scratch,

```

```

286         args.out_uri,
287         video_id,
288         [],
289         "failed",
290         storage_cfg,
291         str(getattr(args, "video_uri", "") or ""),
292     )
293 except Exception as e: # never mask the real failure on an artifact-push hiccup
294     logger.warning("[worker] could not push failure artifacts: %s", e)
295 # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
296 if getattr(args, "done_uri", None):
297     _write_done_marker(
298         args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299     )
300 return rc
301
302 segments: List[dict] = []
303 segmenter_actual = None
304 segmentation_ran = False
305 status = "failed"
306 try:
307     if failures:
308         print(
309             "[worker] validation FAILED: "
310             + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311         )
312         return _fail(2)
313     seg_name = args.segmenter or config.indexing.default_segmenter
314     segmenter_cls = segmenter_registry.get(seg_name)
315     if not segmenter_cls:
316         print(
317             f"[worker] unknown segmenter: {seg_name!r} "
318             f"(have {list(segmenter_registry.list_available())})"
319         )
320         return _fail(2)
321     segmenter_actual = seg_name
322     result = segmenter_cls(db, config).process_video(
323         video_id, local_video, max_frames=args.max_frames
324     )
325     segmentation_ran = True
326     if isinstance(result, Failure):
327         print(f"[worker] segmenter FAILED: {result.message}")
328         return _fail(3)
329     segments = result.value if hasattr(result, "value") else result
330     status = "success"
331     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334     if not segments:
335         detector_impl = (
336             os.environ.get("RALLY_DETECTOR_IMPL")
337             or getattr(config.indexing, "detector_impl", None)
338             or "auto"
339         )
340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "

```

```

345         "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346         "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347         "detections, or the input resolution differs from the tuned proxy
resolution. "
348         "Re-run with -v for the native command + frame counts.",
349         video_id,
350         segmenter_actual,
351         detector_impl,
352     )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(
370         scratch,
371         args.out_uri,
372         video_id,
373         segments,
374         status,
375         storage_cfg,
376         str(getattr(args, "video_uri", "") or ""),
377     )
378
379     print(
380         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
381     )
382     for u in pushed:
383         print("    ", u)
384
385     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
386     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
387     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
388     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
389     if getattr(args, "done_uri", None):
390         _write_done_marker(
391             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
392         )
393     return 0
394
395
396     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
397     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
398     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

```



```

399
400     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
401     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
402     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
403     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
404
405
406     def _label_clip_name(fname: str) -> str:
407         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
408
409         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
410         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
411
412         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
413         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
414         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
415         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
416
417         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
418         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
419         """
420         name = fname.replace(".rallies.csv", "").replace(".csv", "")
421         return _LABEL_DATE_PREFIX.sub("", name)
422
423
424     def _probe_video_fps_width(path: str):
425         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
426
427         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
428         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
429         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
430         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
431         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
432         rather than guesses.
433         """
434         import json
435         import subprocess
436
437         try:
438             out = subprocess.check_output(
439                 [
440                     ffprobe_bin(),
441                     "-v",
442                     "error",
443                     "-select_streams",
444                     "v:0",
445                     "-show_entries",
446                     "stream=r_frame_rate,width",
447                     "-of",
448                     "json",

```

```

449         path,
450     ],
451     stderr=subprocess.DEVNULL,
452 ).decode()
453 st = (json.loads(out).get("streams") or [{}])[0]
454 num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
455 fps = float(num) / float(den or "1")
456 width = float(st.get("width") or 0)
457 if fps > 0 and width > 0:
458     return fps, width
459 except (
460     subprocess.SubprocessError,
461     ValueError,
462     KeyError,
463     OSError,
464     json.JSONDecodeError,
465 ):
466     pass
467 return None
468
469
470 def _resolve_train_detector(args: argparse.Namespace, config: Any):
471     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
472     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
473     box,
474     stub=CPU mock). Returns the runner, or prints an error + returns None.
475
476     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
477     the
478     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
479     has
480     no shuttle detector and is rejected.
481     """
482     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
483
484     seg_cls = segmenter_registry.get(args.segmenter)
485     if not seg_cls:
486         print(
487             f"[worker] unknown segmenter: {args.segmenter!r} "
488             f"(have {list(segmenter_registry.list_available())})"
489         )
490         return None
491     seg = seg_cls(None, config)
492     if not isinstance(seg, TrajectoryHybridSegmenter):
493         print(
494             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
495             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
496         )
497         return None
498     try:
499         runner, _ = seg._resolve_runner(config.indexing)
500     except NotImplementedError as e:
501         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
502         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
503         traceback.
504         print(f"[worker] detector not available: {e}")
505         return None
506     ok, msg = runner.healthcheck()
507     if not ok:
508         print(f"[worker] detector environment not ready: {msg}")
509         return None
510     print(f"[worker] detector ready ({args.segmenter}): {msg}")
511     return runner

```

```

510     def cmd_train(args: argparse.Namespace) -> int:
511         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
512         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
513
514         Two trajectory sources (the train pipeline + harness are identical either way):
515         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
516         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
517         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
518         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
519         is the M3.5 "first real training" path; only the trajectory source flips.
520         """
521         import subprocess
522
523         config = load_config(args.config) if args.config else load_config()
524         _apply_overrides(config, args.set)
525         if not _run_startup_checks(config):
526             return 2
527         storage_cfg = config.storage.model_dump()
528
529         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
530         labels_dir = os.path.join(scratch, "labels")
531         traj_dir = os.path.join(scratch, "trajectories")
532         videos_dir = os.path.join(scratch, "videos")
533         os.makedirs(labels_dir, exist_ok=True)
534         os.makedirs(traj_dir, exist_ok=True)
535
536         corpus = args.corpus_uri.rstrip("/")
537         label_uris = sorted(
538             u
539             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
540             if u.lower().endswith(".csv")
541         )
542         if len(label_uris) < 2:
543             print(
544                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)}"
545                 f"under {corpus}/labels/"
546             )
547             return 2
548
549         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
550         runner = None
551         video_by_stem: dict = {}
552         if args.trajectory_source == "detector":
553             os.makedirs(videos_dir, exist_ok=True)
554             runner = _resolve_train_detector(args, config)
555             if runner is None:
556                 return 2
557             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
558                 vname = storage.parse_uri(vuri).name
559                 if not vname.lower().endswith(_VIDEO_EXTS):
560                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
561                 video_by_stem[os.path.splitext(vname)[0]] = vuri
562
563         print(f"[worker] trajectory source: {args.trajectory_source}")
564         specs: List[dict] = []
565         for uri in label_uris:
566             fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv

```

```

567     name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
568     local_label = storage.fetch(
569         uri, os.path.join(labels_dir, fname), config=storage_cfg
570     )
571     if args.trajectory_source == "detector":
572         vuri = video_by_stem.get(name)
573         if not vuri:
574             print(
575                 f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
576             )
577             continue
578         local_video = storage.fetch(
579             vuri,
580             os.path.join(videos_dir, storage.parse_uri(vuri).name),
581             config=storage_cfg,
582         )
583         video_id = compute_video_id(local_video)
584         # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
585         # wrong fps silently mis-aligns every label, so skip the video if ffmpeg
can't read it.
586         meta = _probe_video_fps_width(local_video)
587         if meta is None:
588             print(
589                 f"[worker] could not probe fps/width for {name!r} (ffmpeg) ?
skipping (won't guess)."
590             )
591             continue
592         fps, frame_width = meta
593         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
594         if not traj:
595             print(
596                 f"[worker] detector produced no trajectory for {name!r} ? skipping."
597             )
598             continue
599         specs.append(
600             {
601                 "name": name,
602                 "trajectory": traj,
603                 "labels": local_label,
604                 "fps": fps,
605                 "frame_width": frame_width,
606             }
607         )
608     else:
609         from backend.pipeline.detectors.stub_runner import (
610             synth_trajectory_from_labels,
611         )
612
613         traj = os.path.join(traj_dir, f"{name}_traj.csv")
614         synth_trajectory_from_labels(
615             local_label, traj, fps=args.fps, frame_width=args.frame_width
616         )
617         specs.append(
618             {
619                 "name": name,
620                 "trajectory": traj,
621                 "labels": local_label,
622                 "fps": args.fps,
623                 "frame_width": args.frame_width,
624             }
625         )
626
627 if len(specs) < 2:
628     print(

```

```

629         f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}]."
630     )
631     return 2
632     manifest_path = os.path.join(scratch, "manifest.json")
633     with open(manifest_path, "w", encoding="utf-8") as f:
634         json.dump(specs, f, indent=2)
635     src_label = (
636         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
637     )
638     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
639
640     out_dir = os.path.join(scratch, "artifacts")
641     cmd = [
642         sys.executable,
643         "-m",
644         "training.gen0.harness",
645         "--manifest",
646         manifest_path,
647         "--out",
648         out_dir,
649         "--version",
650         args.version,
651     ]
652     if args.floor:
653         floor_local = (
654             storage.fetch(
655                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
656             )
657             if "://" in args.floor
658             else args.floor
659         )
660         cmd += ["--floor", floor_local]
661     print("[worker] training:", " ".join(cmd))
662     rc = subprocess.run(cmd).returncode
663     if rc != 0:
664         print(f"[worker] gen0 harness failed (rc={rc})")
665         return rc
666
667     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
668     bundle = os.path.join(out_dir, args.version)
669     out_prefix = args.out_uri.rstrip("/")
670     pushed = []
671     for fn in sorted(os.listdir(bundle)):
672         uri = f"{out_prefix}/weights/{args.version}/{fn}"
673         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
674         pushed.append(uri)
675     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
676     for u in pushed:
677         print("    ", u)
678     return 0
679
680
681 def build_parser() -> argparse.ArgumentParser:
682     p = argparse.ArgumentParser(
683         prog="python -m backend.worker",
684         description="Storage-aware worker: pull ? run pipeline ? push.",
685     )
686     p.add_argument(
687         "-v",
688         "--verbose",
689         action="store_true",
690         help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
691     )

```

```

692     sub = p.add_subparsers(dest="cmd", required=True)
693
694     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
695     pi.add_argument(
696         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
697     )
698     pi.add_argument(
699         "--out-uri",
700         required=True,
701         help="output prefix (outputs/<video_id>/? is appended)",
702     )
703     pi.add_argument(
704         "--segmenter", default=None, help="default: indexing.default_segmenter"
705     )
706     pi.add_argument(
707         "--config", default=None, help="config.json path (default: ./config.json)"
708     )
709     pi.add_argument(
710         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
711     )
712     pi.add_argument("--max-frames", type=int, default=None)
713     pi.add_argument(
714         "--done-uri",
715         default=None,
716         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
717         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
718         "Default-OFF: omit it for the normal local run.",
719     )
720     pi.add_argument(
721         "--set",
722         action="append",
723         default=[],
724         metavar="k=v",
725         help="config override, e.g. --set indexing.skip_ai_handoff=true",
726     )
727     pi.set_defaults(func=cmd_infer)
728
729     pt = sub.add_parser(
730         "train",
731         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
732     )
733     pt.add_argument(
734         "--corpus-uri",
735         required=True,
736         help="prefix containing labels/ (+ videos/ for "
737         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
738     )
739     pt.add_argument(
740         "--out-uri",
741         required=True,
742         help="output prefix (weights/<version>/ is appended)",
743     )
744     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
745     pt.add_argument(
746         "--trajectory-source",
747         choices=["synth", "detector"],
748         default="synth",
749         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
750         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
751         "(needs videos/ in the corpus; honours detector_impl).",
752     )
753     pt.add_argument(
754         "--segmenter",
755         default="wasb_hybrid",
756         help="trajectory hybrid whose detector is used for --trajectory-source

```

```

detector",
757     )
758     pt.add_argument(
759         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
760     )
761     pt.add_argument("--config", default=None)
762     pt.add_argument("--scratch", default=None)
763     pt.add_argument(
764         "--fps",
765         type=float,
766         default=30.0,
767         help="synth trajectory fps (detector probes the video)",
768     )
769     pt.add_argument(
770         "--frame-width",
771         type=float,
772         default=1280.0,
773         help="synth trajectory frame width (detector probes the video)",
774     )
775     pt.add_argument("--set", action="append", default=[], metavar="k=v")
776     pt.set_defaults(func=cmd_train)
777     return p
778
779
780     def _setup_logging(verbose: bool) -> None:
781         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
782         diagnostics are
783         VISIBLE ? the worker previously ran with no logging config, so every
784         segmenter/detector log
785         (detector choice, 'Running native WASB inference', availability failures) was
786         silently dropped.
787         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
788         (the CLI
789         entry), not at import, so importing the module never reconfigures a caller's
790         logging."""
791         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
792         library
793         # configured logging at import), which would leave the level at WARNING and re-
794         suppress the
795         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
796         verbosity.
797         logging.basicConfig(
798             level=logging.DEBUG if verbose else logging.INFO,
799             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
800             datefmt="%H:%M:%S",
801             force=True,
802         )
803
804     def main(argv=None) -> int:
805         args = build_parser().parse_args(argv)
806         _setup_logging(getattr(args, "verbose", False))
807         return args.func(args)
808
809     if __name__ == "__main__":
810         sys.exit(main())
811

```

8. assistant (2026-07-06T17:27:13.195Z)

Now let me check the consumer code paths: orchestration.py (cached_process_result, probe_process_cache), job_worker.py, cloud_run.py, and how main.py handles the Failure vs bare-list. Let me also check the gemini/motion segmenters (the Failure precedent).

9. user (2026-07-06T17:27:15.226Z)

```
230     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
232     ref = storage.parse_uri(uri)
233     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
234         local = storage.fetch(
235             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
236         )
237         with open(local, encoding="utf-8") as f:
238             loaded = json.load(f)
239         return loaded if isinstance(loaded, dict) else None
240
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
243     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
244     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
245     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
246     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
247     _REHYDRATE_LOCKS_GUARD = threading.Lock()
248
249
250     def _rehydrate_lock(video_id: str) -> threading.Lock:
251         with _REHYDRATE_LOCKS_GUARD:
252             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
253
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
257
258         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
259         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
260         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
261         errors report False (? the normal dispatch path with its guards)."""
262         try:
263             existing = db.get_video(video_id)
264             if (
265                 existing
266                 and existing.get("status") == "processed"
267                 and db.query_play_segments(video_id, {})
268             ):
269                 return True
270             sc = _storage_settings(cfg)
271             if not sc or (sc.get("backend") or "local") == "local":
272                 return False
273             output_root = (sc.get("output_root") or "").rstrip("/")
274             if not output_root:
275                 return False
276             report_uri = f"{output_root}/outputs/{video_id}/report.json"
277             if not storage.exists(report_uri, config=sc):
278                 return False
279             report = _read_bucket_json(report_uri, sc)
280             return bool(report and report.get("status") == "success")
281         except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282             logger.warning(
283                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284             )
```



```

285         return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
295         no
296         completed work found (or the check failed) ? the caller dispatches normally. The
297         bucket
298         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
299         so a
300         partial earlier ingest is replaced, never double-counted.
301
302         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
303         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
304         the
305         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
306         CLAIMED
307         job before invoking (writes happen post-claim)."""
308         try:
309             with _rehydrate_lock(video_id):
310                 existing = db.get_video(video_id)
311                 if existing and existing.get("status") == "processed":
312                     segs = db.query_play_segments(video_id, {})
313                     if segs:
314                         return _cached_process_payload(
315                             video_id,
316                             existing.get("validation_results", []),
317                             segs,
318                             requested_compute,
319                         )
320                 sc = _storage_settings(cfg)
321                 if not sc or (sc.get("backend") or "local") == "local":
322                     return None
323                 output_root = (sc.get("output_root") or "").rstrip("/")
324                 if not output_root:
325                     return None
326                 outputs_uri = f"{output_root}/outputs/{video_id}"
327                 report_uri = f"{outputs_uri}/report.json"
328                 if not storage.exists(report_uri, config=sc):
329                     return None
330                 report = _read_bucket_json(report_uri, sc)
331                 if not report or report.get("status") != "success":
332                     return None
333                 if not video_path:
334                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
335                     report
336                     # carries the worker's --video-uri (P3); older reports fall back to the
337                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
338                     # recover, and the next same-md5 submit re-binds the truthful path via
339                     # add_video's upsert.
340                     video_path = str(report.get("video_uri") or "") or _first_object_under(
341                         f"{output_root}/videos/{video_id}/", sc
342                     )
343                 play_wm, cand_wm = db.segment_watermarks(video_id)
344                 db.add_video(video_id, video_path, "processed", [])
345                 n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
346                 if n <= 0:
347                     return None
348                 segments = [
349                     s

```

```

344         for s in db.query_play_segments(video_id, {}):
345             if int(s.get("id", 0)) > play_wm:
346                 ]
347             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348         if not segments:
349             return None

```

10. user (2026-07-06T17:27:15.399Z)

```

1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8      We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9      This is intentionally small and incremental ? we start by applying it to the segmenter
10     and provider layers (the biggest sources of silent [] returns today).
11
12     The types are generic where it makes sense. Callers can still access .value on Success
13     or .message on Failure.
14
15     This module has no dependencies on the rest of the app.
16     """
17
18     from __future__ import annotations
19
20     from dataclasses import dataclass, field
21     from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
23     T = TypeVar("T")
24
25
26     @dataclass
27     class Success(Generic[T]):
28         """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30         value: T
31         warnings: List[str] = field(default_factory=list)
32
33         def is_success(self) -> bool:
34             return True
35
36         def unwrap(self) -> T:
37             """Return the value. Only safe to call if you already know it's a Success."""
38             return self.value
39
40
41     @dataclass
42     class Failure:
43         """Represents a failure with a human-readable message and optional details."""
44
45         message: str
46         details: Optional[dict[str, Any]] = None
47         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49         def is_success(self) -> bool:
50             return False
51
52         def unwrap(self) -> Any:
53             """Raise to make misuse obvious in code that assumes success."""
54             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")

```

```

55
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
62
63     def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64         """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66         - ``Failure``          -> ``(None, failure)``
67         - ``Success(value=v)`` -> ``(v, None)``
68         - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70
71         This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72         that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73         on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74         (Success | Failure | legacy bare value)."""
75         if isinstance(result, Failure):
76             return None, result
77         value = result.value if hasattr(result, "value") else result
78         return value, None
79

```

11. assistant (2026-07-06T17:27:21.383Z)

Now let me check the main.py Failure branch and the job_worker.py / cloud_run.py consumers to understand the full control plane picture. Let me also check the gemini/motion segmenters for the Failure precedent, and look at how fusion_hybrid relates to trajectory_hybrid.

12. user (2026-07-06T17:27:23.059Z)

```

1530         print("\n" + "=" * 40)
1531         print("VALIDATION SUMMARY")
1532         print("=" * 40)
1533         for r in val_results:
1534             status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535             print(f"{status_symbol} {r.validator_name}: {r.message}")
1536         print("=" * 40 + "\n")
1537
1538         if failures:
1539             print("[FAIL] Video failed checks. Indexing halted.")
1540             return
1541
1542         # Index
1543         segmenter_cls = segmenter_registry.get(seg_name)
1544         if not segmenter_cls:
1545             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546             return
1547
1548         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549         segmenter_actual = seg_name
1550         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551         segmenter = segmenter_cls(db_instance, global_config)
1552         result = segmenter.process_video(
1553             video_id, local_video, max_frames=args.max_frames
1554         )
1555         segmentation_ran = True

```

```

1556
1557         if isinstance(result, Failure):
1558             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559             print("[FAIL] Indexing halted for video.")
1560             segments = []
1561         else:
1562             segments = result.value if hasattr(result, "value") else result
1563             print(
1564                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565             )
1566             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568         finally:
1569             # Always emit the per-video observability report. Never raises.
1570             paths = emit_indexer_report(
1571                 db=db_instance,
1572                 video_id=video_id,
1573                 # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                 # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                 # in DB provenance via db.add_video.
1576                 video_path=local_video,
1577                 config=_report_config_for_input(global_config, args.video_path),
1578                 val_results=val_results,
1579                 val_context=val_context,
1580                 segmenter_requested=args.segmenter,
1581                 segmenter_actual=segmenter_actual,
1582                 was_reingest=was_reingest,
1583                 segmentation_ran=segmentation_ran,
1584             )
1585             if paths:
1586                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1589         finally:
1590             storage.cleanup_acquired_local_input(
1591                 args.video_path, local_video, storage_cfg
1592             )
1593         return
1594
1595     # Start Local Server Mode
1596     if args.app or args.server or not args.video_path:
1597         if args.app:
1598             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1600             bind_host = "127.0.0.1"
1601             port = _resolve_app_port(bind_host, args.port)
1602             if port != args.port:
1603                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1604             url = f"http://{bind_host}:{port}/"
1605             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1606             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1607             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1608             print(
1609                 f"[APP] Starting KhelSutra ? opening {url} in your browser "

```

```

1      """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6      re-claimed when due ? with no central scheduler (the DB is the clock).
7
8      The monkeypatch contract these functions rely on (and that tests depend on):
9      every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10     `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11     `backend.main` module object at call-time (`import backend.main as _main`), never as a
12     bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13     ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15     inside the functions also avoids the circular import at module load (main.py imports
this
16     module; this module must not import main at import time).
17     """
18
19     import json
20     import logging
21     import os
22     import threading
23     import traceback
24     from concurrent.futures import ThreadPoolExecutor
25     from typing import Any, Dict, Optional
26
27     logger = logging.getLogger(__name__)
28
29     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32     # already parallelize their AI handoff internally via their own pools.
33     # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34     # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38     # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39     #
40     # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
41     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44     #   - detector timeout_sec kills hung WSL/GPU calls
45     #   - Cloud Run poll has a max_wait_sec bound
46     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47     # A future slice should add a per-job wall-clock timeout (the executor itself
48     # cannot enforce timeouts on the Thread).
49     _job_executor: Optional[ThreadPoolExecutor] = None
50     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55

```

```

56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec``,
106         NOT a failure. The worker persists `next_poll_at` + `progress` (via

```

```

`set_job_waiting`),
108     releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109     is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111     The wiring is additive: the production segmenter path never raises this today (zero
112     behaviour change), so a job runs exactly as before. The hook is what a later slice
113     (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114     """
115
116     def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117         super().__init__(f"job parked for {delay_sec:.0f}s")
118         self.delay_sec = max(0.0, float(delay_sec))
119         self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139             # the job row for analytics). NULL ? server default = pre-picker behaviour.
140             compute=job.get("compute"),
141             force=bool(params.get("force")),
142         )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:

```

```

162         if is_compile:
163             result = _main._execute_compile(
164                 config,
165                 db,
166                 job,
167                 progress=lambda stage: db.set_job_progress(job_id, stage),
168             )
169         else:
170             req = _main._job_to_request(job)
171             result = _main._execute_processing(
172                 config,
173                 db,
174                 req,
175                 progress=lambda stage: db.set_job_progress(job_id, stage),
176             )
177         if result.get("video_id"):
178             db.set_job_video_id(job_id, result["video_id"])
179         db.mark_job_done(job_id, result)
180         if not is_compile:
181             _maybe_record_job_cost(
182                 job, result, config, db
183             ) # M8 cost ledger (default-OFF; process jobs only)
184             _maybe_run_flywheel(
185                 job, result, config
186             ) # M11 data-flywheel (process jobs only)
187     except _main.JobWaiting as w:
188         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190         db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191     except Exception as e:
192         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193         db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196     def _maybe_record_job_cost(
197         job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198     ) -> None:
199         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
202         pricebook, and
203         persist it.
204
205         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
206         so even with
207         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
208         **0** until a
209         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
210         ledger + pricebook
211         + the recording seam; the usage-emission step is separate (the metering hooks on the
212         real run).
213         With the placeholder pricebook the cost is 0 regardless.
214
215         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
216         is the
217         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
218
219         billing_cfg = getattr(config, "billing", None)
220         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
221             return
222         try:
223             usage = (result or {}).get("usage") or {}
224             db.record_job_cost(
225                 job["id"],
226                 usage=usage,

```



```

221         pricebook_version=billing_cfg.pricebook_version,
222         gpu_type=usage.get("gpu_type"),
223         compute_backend=(result or {}).get("compute_backend"),
224         owner_id=job.get("owner_id"),
225         margin=billing_cfg.margin,
226     )
227     logger.info(
228         "Recorded cost for job %s (pricebook=%s).",
229         job["id"],
230         billing_cfg.pricebook_version,
231     )
232 except Exception as e: # never wedge a completed job on bookkeeping
233     logger.warning(
234         "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235     )
236
237
238 def _maybe_run_flywheel(
239     job: Dict[str, Any], result: Dict[str, Any], config: Any
240 ) -> None:
241     """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242     ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243     (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244     land), so this captures **nothing** in production. When activated it captures the
consented job's
245     artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247     Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248     row (carries ``owner_id``); ``result`` is the pipeline output."""
249
250     flywheel_cfg = getattr(config, "flywheel", None)
251     if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252         return
253     try:
254         from backend import flywheel
255
256         flywheel.run_for_job(job, result, config)
257     except Exception as e: # never wedge a completed job on the flywheel
258         logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261 def _drain_due_jobs(config: Any, db: Any) -> int:
262     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267     a single-worker box keeps making progress with no central timer (the DB is the
clock).
268
269     Returns the number of jobs that reached a terminal/parked state this tick.
270     """
271     import backend.main as _main
272
273     ran = 0
274     while True:
275         job = db.claim_due_job()
276         if job is None:
277             break

```

```

277         ran += 1
278         _main._run_claimed_job(job, config, db)
279         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281         # on the next submit/drain. Best-effort ? never raises into the worker.
282         _main._schedule_next_drain_if_waiting(config, db)
283         return ran
284
285
286     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288         self-scheduled job resumes with no further client submit. The drain itself re-checks
289         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:
293             delay = db.seconds_until_next_due()
294         except Exception as e: # pragma: no cover - defensive; never wedge the worker
295             logger.error(f"Could not compute next-due delay: {e}")
296             return
297         if delay is None:
298             return # nothing waiting
299         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
307         import backend.main as _main
308
309         timer = threading.Timer(
310             delay_sec,
311             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312         )
313         timer.daemon = True
314         timer.start()
315
316
317     #: A parked job further out than this is re-checked by a capped wake rather than one
long
318     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319     _MAX_DRAIN_WAKE_SEC = 60.0
320
321
322     #
-----
323     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
324     # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
325     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
326     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
327     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever

```

```

328     # re-dispatching (no double GPU spend).
329
330     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331     _RESUME_POLL_EVERY_SEC = 15.0
332
333
334     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
337         startup,
338         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
339         daemon
340         thread (bounded by the handful of rows a single-instance CP can have in flight);
341         rows are
342         NEVER re-dispatched from here. Returns the number of resumes started.
343
344         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
345         can
346         exist ? the rows get today's honest terminal failure instead."""
347         rows = db.interrupted_remote_process_jobs()
348         if not rows:
349             return 0
350         if not _cloud_dispatch_possible(config):
351             for job in rows:
352                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
353                 logger.warning(
354                     "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
355                     "dispatch to Cloud Run, so no remote worker can be in flight.",
356                     len(rows),
357                 )
358             return 0
359         # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
360         # compute='cloud' always was; NULL means "server default", which is remote ONLY when
361         the
362         # default target itself is cloud_run (on a default-local server with cloud-override
363         # capability, a NULL row ran in-process ? no remote worker exists to resume).
364         default_is_cloud = (
365             getattr(getattr(config, "deployment", None), "compute_target", "") ==
366             "cloud_run"
367         )
368         started = 0
369         for job in rows:
370             # Same normalization the execution path applies (_maybe_dispatch_remote
371             lowercases
372             # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
373             at run.
374             requested = (str(job.get("compute") or "")).strip().lower()
375             if requested != "cloud" and not default_is_cloud:
376                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
377                 continue
378             t = threading.Thread(
379                 target=_resume_one_remote_job,
380                 args=(config, db, dict(job)),
381                 daemon=True,
382                 name=f"resume-{str(job['id'])[:8]}",
383             )
384             t.start()
385             started += 1
386         if started:
387             logger.warning(
388                 "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
389                 job(s).",
390                 started,
391             )
392         return started

```

```

384
385
386 def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387     """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
388     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389     fail the job HONESTLY from what the bucket says. Never dispatches."""
390     from backend import orchestration, storage
391     from backend.infrastructure.execution_policy import (
392         ExecutionPolicy,
393         PolicyTimeout,
394         poll_until,
395     )
396
397     job_id = str(job["id"])
398     video_id = str(job["video_id"])
399     try:
400         raw_params = job.get("params")
401         params = (
402             raw_params
403             if isinstance(raw_params, dict)
404             else json.loads(raw_params or "{}")
405         )
406         if params.get("force"):
407             # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
408             # but a forced job must NEVER complete from the bucket cache ? force
deliberately
409             # bypasses it, and nothing proves the report belongs to the forced run
rather
410             # than an older completed one.
411             db.mark_job_failed(
412                 job_id,
413                 "control plane restarted during a forced reprocess; the cached result "
414                 "cannot be trusted for force=true ? re-submit with force",
415             )
416             return
417         if (str(job.get("compute") or "").strip().lower() == "local":
418             # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
419             # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
420             db.mark_job_failed(job_id, "interrupted by server restart")
421             return
422         sc = orchestration._storage_settings(config)
423         output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424         if not sc or not output_root:
425             db.mark_job_failed(
426                 job_id,
427                 "interrupted by server restart (no output bucket configured to recover
from)",
428             )
429             return
430         report_uri = f"{output_root}/outputs/{video_id}/report.json"
431         max_wait = float(
432             getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433             or 3900.0
434         )
435         policy = ExecutionPolicy(
436             poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437             op_timeout_sec=120.0,
438             max_wait_sec=max_wait,
439         )

```

```

440         db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441     try:
442         poll_until(
443             lambda: True if storage.exists(report_uri, config=sc) else None,
444             policy,
445             label=f"resume-{job_id[:8]}",
446             logger=logger,
447             # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448             on_tick=lambda elapsed: db.set_job_progress(
449                 job_id,
450                 "recovering: waiting for the in-flight cloud run ? "
451                 f"{int(elapsed) // 60} min",
452             ),
453         )
454     except PolicyTimeout:
455         db.mark_job_failed(
456             job_id,
457             "control plane restarted mid-run and the cloud worker's result never "
458             "appeared within the poll budget; re-submit the video (a finished result
"
459             "re-hydrates instantly)",
460         )
461     return
462     payload = orchestration.cached_process_result(
463         db, config, video_id, str(job.get("video_path") or "")
464     )
465     if payload is not None:
466         db.mark_job_done(job_id, payload)
467         logger.warning(
468             "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469             job_id,
470             video_id,
471         )
472     return
473     # The report exists but did not realize into a usable result ? surface its
verdict.
474     status = "unreadable"
475     try:
476         report = orchestration._read_bucket_json(report_uri, sc) or {}
477         status = str(report.get("status") or "unreadable")
478     except Exception: # noqa: BLE001 ? the failure message is best-effort
479         pass
480     db.mark_job_failed(
481         job_id,
482         f"cloud worker finished without a usable result after a control-plane
restart "
483         f"(report status: {status}); re-submit the video",
484     )
485     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
486         logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487     try:
488         db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489     except Exception: # noqa: BLE001
490         pass
491

```

14. assistant (2026-07-06T17:27:26.849Z)

This is the critical consumer path. Now I understand the full picture. Let me check the fusion_hybrid segmenter (it subclasses TrajectoryHybridSegmenter and is worker-dispatchable), and confirm the wf294 Phase-3 all-handoffs-fail scenario. Let me also look at cloud_run.py's run_infer.

15. user (2026-07-06T17:27:27.491Z)

```
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\_keyhint.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
```

16. user (2026-07-06T17:27:28.792Z)

```
1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6      --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10     cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11     job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12     worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13     the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14     3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17     The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20     The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22     owner-verified on the M7 real run.
23     """
24
25     from __future__ import annotations
26
27     import json
28     import logging
29     import os
30     import tempfile
31     import uuid
32     from abc import ABC, abstractmethod
33     from typing import Any, Callable, List, Optional
34
35     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36     from backend.infrastructure.execution_policy import (
```

```

37         DEFAULT_POLICY,
38         ExecutionPolicy,
39         PolicyTimeout,
40         poll_until,
41     )
42     from backend.storage import fetch, get_storage, parse_uri
43
44     LOG = logging.getLogger("compute.cloud_run")
45
46     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
47     # branch.
48     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
49     # runner.)
50     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
51         "deployment.compute_target=local_gpu",
52         "indexing.detector_impl=native",
53     )
54
55     class CloudRunJobClient(ABC):
56         """Triggers a Cloud Run **Job** execution with per-execution container arg
57         overrides.
58
59         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
60         implementation
61         overrides the job's container ``args`` for this execution and starts it."""
62
63         @abstractmethod
64         def run_job(
65             self,
66             job_name: str,
67             argv: List[str],
68             *,
69             region: str,
70             project: Optional[str] = None,
71         ) -> str:
72             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
73             return an
74             execution id/name (for logs). Does NOT block on completion (the caller bucket-
75             polls)."""
76
77         @abstractmethod
78         def cancel_execution(self, execution: str) -> None:
79             """Cancel a running execution by the name ``run_job`` returned. Called when the
80             bucket-poll
81             times out so the abandoned execution doesn't keep billing the GPU until
82             ``--task-timeout``.
83             May raise ? the caller treats every failure as best-effort (the original poll
84             timeout is
85             NEVER masked by a cancel error)."""
86
87     class GoogleCloudRunJobClient(CloudRunJobClient):
88         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
89         run; CI uses
90         a fake, so this code path is never exercised in tests (the SDK isn't a test
91         dependency)."""
92
93         def run_job(
94             self,
95             job_name: str,
96             argv: List[str],
97             *,
98             region: str,
99             project: Optional[str] = None,

```

```

91         ) -> str:
92             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95             if not project:
96                 raise RuntimeError(
97                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                     "to resolve the job path; set it on the control plane (see
get_compute_backend).")
99                 )
100             try:
101                 from google.cloud import run_v2 # type: ignore[attr-defined]
102             except Exception as exc: # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency).")
106             ) from exc
107             client = (
108                 run_v2.JobsClient()
109             ) # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]
115             )
116             op = client.run_job(
117                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
118             )
119             return (
120                 getattr(op, "metadata", None)
121                 and getattr(op.metadata, "name", "")
122                 or "execution"
123             )
124
125     def cancel_execution(self, execution: str) -> None:
126         # run_job falls back to the literal string "execution" when the operation
metadata carries
127         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
128         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
129         # google-cloud-run is absent).
130         if "/executions/" not in (execution or ""):
131             raise RuntimeError(
132                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
133                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
134                 "one from the operation metadata, so there is nothing to cancel by
name).")
135             )
136         try:
137             from google.cloud import run_v2 # type: ignore[attr-defined]
138         except Exception as exc: # pragma: no cover - real SDK not present in CI
139             raise RuntimeError(
140                 "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
141                 "control plane (it is intentionally NOT a CI/test dependency).")
142             ) from exc

```



```

143         client = (
144             run_v2.ExecutionsClient()
145         ) # pragma: no cover - exercised only on a real run
146         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
147
148
149     class CloudRunCompute(ComputeBackend):
150         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
151
152         def __init__(
153             self,
154             *,
155             run_client: CloudRunJobClient,
156             job_name: str,
157             region: str,
158             project: Optional[str] = None,
159             storage_config: Optional[dict] = None,
160             policy: ExecutionPolicy = DEFAULT_POLICY,
161             token: Optional[str] = None,
162             container_overrides: Optional[tuple[str, ...]] = None,
163         ) -> None:
164             self._client = run_client
165             self._job_name = job_name
166             self._region = region
167             self._project = project
168             self._storage_config = storage_config or {}
169             self._policy = policy
170             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172             self._token = token
173             self._container_overrides = (
174                 _DEFAULT_CONTAINER_OVERRIDES
175                 if container_overrides is None
176                 else tuple(container_overrides)
177             )
178
179         def run_infer(
180             self,
181             spec: ComputeJobSpec,
182             *,
183             on_wait: "Callable[[float], None] | None" = None,
184         ) -> ComputeResult:
185             out_root = spec.out_uri.rstrip("/")
186             token = self._token or uuid.uuid4().hex
187             done_uri = f"{out_root}/_dispatch/{token}.done"
188             argv: List[str] = [
189                 "infer",
190                 "--video-uri",
191                 spec.video_uri,
192                 "--out-uri",
193                 spec.out_uri,
194                 "--done-uri",
195                 done_uri,
196             ]
197             if spec.segmenter:
198                 argv += ["--segmenter", spec.segmenter]
199             for kv in (*spec.config_overrides, *self._container_overrides):
200                 argv += ["--set", kv]
201
202             execution = self._client.run_job(
203                 self._job_name, argv, region=self._region, project=self._project
204             )
205             LOG.info(

```

```

206         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
207         self._job_name,
208         execution,
209         done_uri,
210     )
211
212     try:
213         marker = poll_until(
214             lambda: self._read_marker(done_uri),
215             self._policy,
216             label="cloud-run-infer",
217             logger=LOG,
218             # Live heartbeat (#482): each still-waiting tick reaches the caller so
219             # job.progress moves during the whole GPU run instead of freezing.
220             on_tick=on_wait,
221         )
222     except PolicyTimeout as timeout_exc:
223         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
224         video_id = str(marker.get("video_id", ""))
225         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
226         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
227         rc = int(marker.get("returncode", 1))
228         if not video_id:
229             LOG.warning(
230                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
231                 "video_id) ? treating as failure",
232                 self._job_name,
233             )
234             rc = rc or 1
235         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
236         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
237         LOG.info(
238             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
239         )
240         return ComputeResult(
241             returncode=rc,
242             outputs_uri=outputs_uri,
243             video_id=video_id,
244             report=report,
245             execution=str(execution or ""),
246         )
247
248     def _handle_poll_timeout(
249         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
250     ) -> dict:
251         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
252
253         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
job's own
254         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
255         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
256         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
257         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
258         execution name so the operator can verify the state in the GCP console. A cancel
failure
259         NEVER masks the original timeout."""
260         try:
261             late = self._read_marker(done_uri)
262         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence

```

```

? failure
263         LOG.warning(
264             "cloud-run: final post-timeout marker check failed ? treating as
absent",
265             exc_info=True,
266         )
267         late = None
268     if late is not None:
269         LOG.warning(
270             "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
271             "? rescuing the completed result",
272             self._job_name,
273             execution,
274         )
275         return late
276     try:
277         self._client.cancel_execution(execution)
278     except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
279         cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
280         LOG.warning(
281             "cloud-run: best-effort cancel of execution=%s failed: %s",
282             execution,
283             cancel_exc,
284             exc_info=True,
285         )
286     else:
287         cancel_note = "cancel requested"
288         LOG.info("cloud-run: requested cancel of execution=%s", execution)
289         raise PolicyTimeout(
290             f"{timeout_exc} [cloud-run job={self._job_name} "
291             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
292             "state in the GCP console]"
293         ) from timeout_exc
294
295     # --- storage-backed completion signal (bucket-poll) ---
296     def _read_marker(self, done_uri: str) -> Optional[dict]:
297         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
298         ref = parse_uri(done_uri)
299         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
300             return None
301         return (
302             self._read_json(done_uri) or {}
303         ) # empty dict still terminates the poll (present == done)
304
305     def _read_json(self, uri: str) -> dict:
306         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
307         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
308         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
309         try:
310             ref = parse_uri(uri)
311             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
312                 dst = os.path.join(td, ref.name or "obj.json")
313                 local = fetch(uri, dst, config=self._storage_config)
314                 with open(local, "r", encoding="utf-8") as f:
315                     data: Any = json.load(f)
316                 return data if isinstance(data, dict) else {}
317         except Exception:
318             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)

```

```
319         return {}
320
```

17. assistant (2026-07-06T17:27:34.854Z)

Now let me read the fusion_hybrid segmenter (the third worker-dispatchable trajectory hybrid), and check the gemini/motion precedent to confirm how they use Failure. Fusion is critical because it subclasses TrajectoryHybridSegmenter and may override process_video.

18. user (2026-07-06T17:27:36.362Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13     torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28
29     from typing import Any, Dict, List, Optional, Tuple
30
31     from backend.config.models import FusionConfig, IndexingConfig
32     from backend.pipeline.detectors import fusion_features as ff
33     from backend.pipeline.detectors import rally_gate
34     from backend.pipeline.detectors.base import DetectorRunner
35     from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36     from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
37     from backend.pipeline.segmenters.base import segmenter_registry
38     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
39
40
41     class FusionSegmenter(TrajectoryHybridSegmenter):
42         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
43
```

```

44     family = "fusion"
45     display_label = "Fusion"
46
47     def __init__(self, db: Any, config: Any):
48         super().__init__(db, config)
49         # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
50         self._person: Optional[Any] = None
51         # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
52         # over the whole trajectory once per candidate window (it depends only on
`points`).
53         self._axis_cache_key: Optional[int] = None
54         self._axis_cache: Optional[tuple] = None
55
56     @property
57     def name(self) -> str:
58         return "fusion_hybrid"
59
60     @property
61     def description(self) -> str:
62         return (
63             "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
64             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and
the "
65             "AI provider sets semantic boundaries."
66         )
67
68     def _build_runner(
69         self, idx_cfg: IndexingConfig
70     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
71         if isinstance(idx_cfg, dict):
72             idx_cfg = IndexingConfig(**idx_cfg)
73         substrate = idx_cfg.fusion.substrate
74         if substrate == "tracknet":
75             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
76             return TrackNetRunner(cfg), {
77                 "weights_path": cfg.weights_path,
78                 "queue_length": cfg.queue_length,
79                 "fusion_substrate": "tracknet",
80             }
81         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
82         return WasbRunner(wcfg), {
83             "weights_path": wcfg.weights_path,
84             "fusion_substrate": "wasb",
85         }
86
87     def _build_native_runner(
88         self, idx_cfg: IndexingConfig
89     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
90         # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
91         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
92         substrate = idx_cfg.fusion.substrate
93         if substrate != "wasb":
94             raise NotImplementedError(
95                 "detector_impl='native' supports the WASB substrate only ? set "
96                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet)."
97             )
98         from backend.pipeline.detectors.native_wasb_runner import (
99             NativeWasbConfig,
100             NativeWasbRunner,
101         )

```

```

102
103         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
104         return NativeWasbRunner(cfg), {
105             "weights_path": cfg.weights_path,
106             "detector_impl": "native",
107             "fusion_substrate": "wasb",
108             "device": cfg.device,
109         }
110
111     def _enrich_candidate_stats(
112         self,
113         cw: Dict[str, Any],
114         points: List[Any],
115         fps: float,
116         frame_width: float,
117         video_path: str,
118         idx_cfg: IndexingConfig,
119     ) -> Dict[str, Any]:
120         stats = super()._enrich_candidate_stats(
121             cw, points, fps, frame_width, video_path, idx_cfg
122         )
123         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
124         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
125         # estimated
126         # over the whole trajectory by rally_gate (camera-agnostic); reused across
127         # windows.
128         gated = rally_gate.gate_windows(
129             points,
130             [(cw["start"], cw["end"])],
131             fps,
132             min_crossings=0,
133             estimate=self._axis_estimate(points),
134         )
135         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
136
137         fcfg = idx_cfg.fusion
138         if fcfg.cue_flags.get("person", False):
139             stats.update(
140                 self._person_features(cw, points, fps, frame_width, video_path, fcfg)
141             )
142         return stats
143
144     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
145     NEXT_STEPS 2026-06-14):
146     # Person features (from _person_features when cue_flags.person) are now persisted
147     # for the harness
148     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
149     # (compute only
150     # for shuttle-seeded windows + padding, not whole video) is already in place via the
151     # lazy
152     # self._person + windowed detections_per_frame. Next: actual harness run + lift
153     # measurement
154     # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
155     # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
156     # discipline.
157
158     def _axis_estimate(self, points: List[Any]) -> tuple:
159         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
160         it's
161         computed once per video, not once per candidate window (it depends only on
162         points)."""
163         key = id(points)
164         if self._axis_cache_key != key or self._axis_cache is None:
165             self._axis_cache_key = key

```

```

156         self._axis_cache = rally_gate.estimate_play_axis(points)
157     est = self._axis_cache
158     assert est is not None
159     return est
160
161     def _person_features(
162         self,
163         cw: Dict[str, Any],
164         points: List[Any],
165         fps: float,
166         frame_width: float,
167         video_path: str,
168         fcfg: "FusionConfig",
169     ) -> Dict[str, float]:
170         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
171         frame range and compute the scale/translation-invariant person features. Lazily
builds
172         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
173         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
174         from backend.pipeline.detectors.person_detector import PersonDetector
175
176         if self._person is None:
177             self._person = PersonDetector(self.config)
178
179         frame_skip = self.config.indexing.yolo_frame_skip
180         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
181         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
182         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
183         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
184         fw = det["frame_width"] or frame_width
185         fh = det["frame_height"]
186         # If we can't trust the frame dims, every normalised feature would silently
collapse
187         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
188         if fw <= 0 or fh <= 0:
189             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
190
191         poly = fcfg.court_polygon
192         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
193         net = (fcfg.net_axis, float(fcfg.net_position))
194         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
195         return ff.compute_person_features(
196             det["frames"], shuttle_in, fw, fh, court_polygon=polygon, net=net
197         )
198
199     def _select_for_handoff(
200         self,
201         windows: List[Dict[str, Any]],
202         window_stats: List[Dict[str, Any]],
203         idx_cfg: IndexingConfig,
204     ) -> List[int]:
205         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
206         (0) keep every window ? identical to the substrate. Persisted candidates are
207         untouched (full superset).
208
209         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
210         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory

```

```

211         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
212         set rather than emit a video with zero rallies. This makes Gate A/B strictly
213         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
214         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
215         below it always wins on an otherwise-empty video. Observed on real footage
216         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
217         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
218         fcfg = idx_cfg.fusion
219         min_crossings = int(fcfg.min_crossings)
220         min_players = int(fcfg.min_players)
221         person_on = bool(fcfg.cue_flags.get("person", False))
222
223         keep: List[int] = []
224         for i, st in enumerate(window_stats):
225             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
226                 continue
227             if (
228                 person_on
229                 and min_players > 0
230                 and st.get("players_in_court", 0.0) < min_players
231             ):
232                 continue
233             keep.append(i)
234
235         if not keep and window_stats:
236             # Gating emptied a non-empty video ? fall back to the ungated set.
237             return list(range(len(window_stats)))
238         return keep
239
240
241     segmenter_registry.register(FusionSegmenter)
242

```

19. user (2026-07-06T17:27:37.037Z)

```

1     import concurrent.futures
2     import logging
3     import os
4     import subprocess
5     import time
6     from typing import Any, Callable, Dict, List, Tuple
7
8     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9     from backend.providers import provider_registry
10    from backend.reporting import run_signals
11    from backend.results import (
12        Failure, # Finding B: an unrunnable config (missing key/unknown sport/provider) ? a
loud Failure, not a silent 0-rally "success"
13    )
14    from backend.sports import sport_registry
15    from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
16        ffmpeg_bin,
17        ffmpeg_timeout_sec,
18        timeout_label,
19    )
20    from backend.utils.paths import output_base
21    from backend.utils.video import extract_video_chunk, get_video_duration
22
23    logger = logging.getLogger(__name__)
24
25

```



```

26     def _chunk_workdir(video_id: str, base: str = "output") -> str:
27         """Per-video chunk staging dir under <base>/chunks/<video_id> (`base` default
28         ``"output"``
29         keeps the prior literal byte-identical; the caller passes the configured output base
30         staging follows --output-dir ? COMPUTE_DECOUPLED_SERVING M3).
31         Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
32         otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
33         in-flight work. A per-video subdir means each run only ever creates/cleans its own.
34         """
35         return os.path.join(base, "chunks", video_id)
36
37
38     def _parse_time(val) -> float:
39         """Converts a timestamp value to total float seconds.
40         Handles float/int pass-through, plain numeric strings, and
41         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
42         when unexpected formats are encountered."""
43         if isinstance(val, (int, float)):
44             return float(val)
45         if isinstance(val, str):
46             val = val.strip()
47             if ":" in val:
48                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
49                 # Convert it, but warn so the issue is visible in logs.
50                 parts = val.split(":")
51                 parts.reverse()
52                 total = 0.0
53                 for i, p in enumerate(parts):
54                     try:
55                         total += float(p) * (60**i)
56                     except ValueError:
57                         pass
58                 logger.warning(
59                     f"Timestamp '{val}' was in colon-separated format (MM:SS/HH:MM:SS)
instead of decimal seconds. Converted to {total:.2f}s."
60                 )
61                 return total
62             try:
63                 return float(val)
64             except ValueError:
65                 logger.warning(
66                     f"Could not parse timestamp string '{val}' as a number. Defaulting to
0.0."
67                 )
68                 return 0.0
69             logger.warning(
70                 f"Unexpected timestamp type {type(val).__name__} for value '{val}'. Defaulting
to 0.0."
71             )
72             return 0.0
73
74
75     class GeminiPlaySegmenter(BasePlaySegmenter):
76         @property
77         def name(self) -> str:
78             return "gemini"
79
80         @property
81         def description(self) -> str:
82             return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
83
84         def _resolve_provider(self, model_name: str):

```

```

85         """Resolve the sport profile, provider, and API key (the process_video
preamble).
86
87         Returns (sport_profile, provider) on success, or a ``Failure`` (with a user-
facing
88         message) when the run cannot proceed ? an unknown sport, a MISSING API KEY, or
an
89         unknown provider. The caller propagates the Failure so the job is marked
**failed**
90         with a reason, instead of a silent 0-rally "success" that looks like an empty
match
91         (Finding B ? the loud-failures bar). A genuine "ran, found nothing" is
unaffected.
92         """
93         # Get sport profile config (config is always typed AppConfig ?
_normalize_config()
94         # in base.py converts plain dicts passed by tests at init time)
95         sport_name = self.config.sport
96         try:
97             sport_profile = sport_registry.get(sport_name)
98         except KeyError as e:
99             logger.error(str(e))
100             return Failure(str(e))
101
102         # Get AI provider and model config
103         provider_name = self.config.ai_provider
104
105         # Get appropriate API key based on the provider
106         api_key_env_var = f"{provider_name.upper()}_API_KEY"
107         api_key = os.environ.get(api_key_env_var)
108         if not api_key:
109             from backend.pipeline.segmenters._keyhint import api_key_hint
110
111             msg = (
112                 f"{api_key_env_var} is not set ? the {provider_name} segmenter cannot
run. "
113                 f"Set your API key. " + api_key_hint(api_key_env_var)
114             )
115             logger.error(msg)
116             return Failure(msg)
117
118         try:
119             provider = provider_registry.get(
120                 provider_name, api_key=api_key, model_name=model_name
121             )
122         except KeyError as e:
123             logger.error(str(e))
124             return Failure(str(e))
125
126         return sport_profile, provider
127
128     def _maybe_trim_video(
129         self, video_id: str, video_path: str
130     ) -> Tuple[str, Callable[[], None]]:
131         """Optionally trim the video to indexing.debug_duration seconds (debug feature).
132
133         Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
134         temp trimmed file created here.
135         """
136         # A: Check for debug_duration to trim the video first
137         debug_duration = self.config.indexing.debug_duration
138         video_to_upload = video_path
139         is_trimmed = False
140         temp_trimmed_path = None

```

```

141
142         if debug_duration:
143             logger.info(
144                 f"DEBUG FEATURE: Trimming video to first {debug_duration} seconds for
fast processing..."
145             )
146             base = output_base(self.config)
147             temp_trimmed_path = os.path.join(base, f"temp_trimmed_{video_id}.mp4")
148             os.makedirs(base, exist_ok=True)
149             if os.path.exists(temp_trimmed_path):
150                 try:
151                     os.remove(temp_trimmed_path)
152                 except Exception:
153                     pass
154
155             cmd = [
156                 ffmpeg_bin(),
157                 "-y",
158                 "-ss",
159                 "0",
160                 "-t",
161                 str(debug_duration),
162                 "-i",
163                 video_path,
164                 "-c",
165                 "copy",
166                 temp_trimmed_path,
167             ]
168             logger.debug(f"Running command: {' '.join(cmd)}")
169             try:
170                 timeout = ffmpeg_timeout_sec()
171                 subprocess.run(
172                     cmd,
173                     check=True,
174                     stdout=subprocess.DEVNULL,
175                     stderr=subprocess.DEVNULL,
176                     timeout=timeout,
177                 )
178                 logger.info(
179                     f"Trimmed video successfully created at: {temp_trimmed_path}"
180                 )
181                 video_to_upload = temp_trimmed_path
182                 is_trimmed = True
183             except subprocess.TimeoutExpired:
184                 logger.warning(
185                     "Debug-duration ffmpeg trim timed out after %s. Falling back to full
video.",
186                     timeout_label(ffmpeg_timeout_sec()),
187                 )
188             except Exception as e:
189                 logger.warning(
190                     f"Failed to trim video using ffmpeg: {e}. Falling back to full
video."
191                 )
192
193             # Helper to clean up local temp files
194             def cleanup_temp_files():
195                 if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
196                     try:
197                         os.remove(temp_trimmed_path)
198                     except Exception:
199                         pass
200
201             return video_to_upload, cleanup_temp_files
202

```

```

203     def _process_chunked(
204         self,
205         video_id: str,
206         video_to_upload: str,
207         sport_profile,
208         provider,
209         video_duration: float,
210         chunk_duration: float,
211         chunk_overlap: float,
212         ai_scale_width: int,
213         chunks_dir: str,
214         cleanup_temp_files: Callable[[], None],
215     ) -> List[Dict[str, Any]]:
216         """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
217         analyze them in parallel, offset timestamps back to full-video coordinates."""
218         ai_segments: List[Dict[str, Any]] = []
219         step = chunk_duration - chunk_overlap
220         chunk_starts = []
221         t = 0.0
222         while t < video_duration:
223             chunk_starts.append(t)
224             t += step
225         num_chunks = len(chunk_starts)
226         logger.info(
227             f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping
chunks "
228             f"({chunk_duration}s each, {chunk_overlap}s overlap)."
229         )
230
231         os.makedirs(chunks_dir, exist_ok=True)
232
233         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:
234             chunk_end = min(chunk_start + chunk_duration, video_duration)
235             actual_chunk_dur = chunk_end - chunk_start
236             chunk_label = f"Chunk {ci + 1}/{num_chunks}"
237             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
238
239             # Extract chunk with ffmpeg
240             logger.info(
241                 f"Extracting {chunk_label}: {chunk_start:.1f}s - {chunk_end:.1f}s
({actual_chunk_dur:.1f}s)"
242             )
243             success = extract_video_chunk(
244                 video_to_upload,
245                 chunk_start,
246                 actual_chunk_dur,
247                 chunk_path,
248                 reencode=ai_scale_width > 0,
249                 scale_width=ai_scale_width if ai_scale_width > 0 else None,
250             )
251             if not success:
252                 logger.error(f"Failed to extract {chunk_label}. Skipping.")
253                 return chunk_label, [], {}
254
255             # Measure chunk size
256             try:
257                 chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
258             except Exception:
259                 chunk_size_mb = 0.0
260
261             # Get prompt from sport profile
262             prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
263
264             # Process this chunk through the provider
265             chunk_segments, metrics = provider.analyze_video(

```

```

266         chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
267     )
268     metrics["file_size_mb"] = chunk_size_mb
269     metrics["segments_found"] = len(chunk_segments)
270
271     # B2: surface provider failures so callers can distinguish "API error"
272     # from "genuinely no rallies in this chunk" (both return [] today).
273     if metrics.get("error"):
274         logger.warning(
275             "[%s] provider error ? chunk %d skipped: %s",
276             chunk_label,
277             ci,
278             metrics["error"],
279         )
280
281     # Offset timestamps back to full-video coordinates
282     for segment in chunk_segments:
283         if "start_time" in segment:
284             try:
285                 segment["start_time"] = (
286                     float(segment["start_time"]) + chunk_start
287                 )
288             except (ValueError, TypeError):
289                 pass
290         if "end_time" in segment:
291             try:
292                 segment["end_time"] = float(segment["end_time"]) + chunk_start
293             except (ValueError, TypeError):
294                 pass
295
296     logger.info(f"{chunk_label} returned {len(chunk_segments)} play segments.")
297
298     # Clean up chunk file
299     try:
300         os.remove(chunk_path)
301     except Exception:
302         pass
303
304     return chunk_label, chunk_segments, metrics
305
306     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same knob
307     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
308     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns a
309     # misleading "credits depleted" 429 under 5 simultaneous uploads.
310     all_metrics = {}
311     max_workers = max(1, int(self.config.indexing.ai_max_workers))
312     logger.info(
313         "Starting parallel processing with up to %d concurrent workers...",
314         max_workers,
315     )
316     t_chunking_start = time.time()
317     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
318         futures = [
319             executor.submit(process_single_chunk, ci, start)
320             for ci, start in enumerate(chunk_starts)
321         ]
322         for future in concurrent.futures.as_completed(futures):
323             label, segments, metrics = future.result()
324             ai_segments.extend(segments)
325             if metrics:
326                 all_metrics[label] = metrics
327
328     # Oversize refusals must be LOUD: each one is a time range with NO analysis,
329     # so "0 segments / success" here would be a silent failure, not "no rallies".
330     oversize_skips = sum(

```

```

331         1 for m in all_metrics.values() if m.get("oversize_skipped")
332     )
333     if oversize_skips:
334         logger.error(
335             f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload cap
and were "
336             f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges.
"
337             f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
338             f"downscaled before upload."
339         )
340         run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
341
342     # generate() give-ups (per-attempt timeout / 503 bursts exhausting retries) are
the SAME
343     # class of silent failure: the chunk's time range gets NO analysis, so "0
segments" there is
344     # "we don't know", not "no rallies". Surface it loudly + count it so a partial
reel isn't
345     # mistaken for a complete one (observed live: a degraded model alias / capacity
503s dropped
346     # whole chunks while the run still "succeeded").
347     generate_fails = sum(
348         1 for m in all_metrics.values() if m.get("generate_failed")
349     )
350     if generate_fails:
351         logger.error(
352             f"{generate_fails}/{num_chunks} chunks FAILED generate (timeout/503
after retries) ? "
353             f"those time ranges have NO analysis; the result is INCOMPLETE, not 'no
rallies'. "
354             f"Re-run (transient capacity), or pin a healthy indexing model if the
alias is degraded."
355         )
356         run_signals.incr(video_id, "generate_failed_chunks", generate_fails)
357
358     # Print Summary Log
359     summary_lines = [
360         "",
361         "=" * 60,
362         "                GEMINI CHUNKING SUMMARY",
363         "=" * 60,
364         f" Total time: {time.time() - t_chunking_start:.1f}s",
365         f" Total chunks: {num_chunks}",
366         f" Total play segments found: {len(ai_segments)}",
367         "-" * 60,
368         f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
369         "-" * 60,
370     ]
371     for i in range(num_chunks):
372         lbl = f"Chunk {i + 1}/{num_chunks}"
373         m = all_metrics.get(lbl, {})
374         sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
375         upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
376         prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
377         gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
378         seg = m.get("segments_found", 0) if m else 0
379         summary_lines.append(
380             f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} | {gen:<8} | {seg}"
381         )
382     summary_lines.append("=" * 60)
383     logger.info("\n".join(summary_lines))
384

```

```

385         # Clean up local temp files
386         cleanup_temp_files()
387         try:
388             os.rmdir(chunks_dir)
389         except Exception:
390             pass
391
392         logger.info(
393             f"All chunks processed. {len(ai_segments)} total play segments before
deduplication."
394         )
395
396         return ai_segments
397
398     def _process_single(
399         self,
400         video_id: str,
401         video_to_upload: str,
402         sport_profile,
403         provider,
404         video_duration: float,
405         cleanup_temp_files: Callable[[], None],
406     ) -> List[Dict[str, Any]]:
407         """--- SINGLE-FILE PROCESSING (short videos) --- analyze the whole video in one
call."""
408         try:
409             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
410         except Exception:
411             file_sz = 0.0
412
413         # Get prompt from sport profile
414         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
415         ai_segments, metrics = provider.analyze_video(
416             video_to_upload, prompt, chunk_duration=video_duration, label="SingleFile"
417         )
418         if metrics.get("oversize_skipped"):
419             logger.error(
420                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and was "
421                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
422             )
423             run_signals.incr(video_id, "oversize_clips_skipped", 1)
424         elif metrics.get("error"):
425             logger.error("SingleFile provider error: %s", metrics["error"])
426         summary_lines = [
427             "",
428             "=" * 50,
429             "          GEMINI SINGLE FILE SUMMARY",
430             "=" * 50,
431             f" File Size: {file_sz:.1f} MB",
432             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
433             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
434             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
435             f" Play segments found: {len(ai_segments)}",
436             "=" * 50,
437         ]
438         logger.info("\n".join(summary_lines))
439         cleanup_temp_files()
440
441         return ai_segments
442
443     def _validate_and_dedupe(
444         self, ai_segments: List[Dict[str, Any]], video_duration: float
445     ) -> List[Dict[str, Any]]:
446         """Parse timestamps, validate/clamp candidates, and resolve overlaps (keep

```

```

longer)."""
447         logger.info(
448             f"Processing {len(ai_segments)} play segments through validation..."
449         )
450
451     # 7. Parse timestamps and build candidate segment list
452     idx_cfg = self.config.indexing
453     min_segment_duration = idx_cfg.min_segment_duration
454     max_segment_duration = 90.0 # Singles play segments almost never exceed this
455
456     candidates = []
457     for idx, segment in enumerate(ai_segments):
458         start = _parse_time(segment.get("start_time", 0.0))
459         end = _parse_time(segment.get("end_time", 0.0))
460         if start >= end:
461             logger.info(
462                 f"Skipping segment #{idx + 1}: start_time ({start:.2f}s) >= end_time
({end:.2f}s)"
463             )
464             continue
465             candidates.append({"idx": idx, "start": start, "end": end, "raw": segment})
466
467     # 8. Post-hoc validation pass
468     logger.info(
469         f"Running validation on {len(candidates)} candidate play segments..."
470     )
471     validated = []
472     rejected = []
473
474     for c in candidates:
475         label = f"Segment #{c['idx'] + 1} [{c['start']:.2f}s - {c['end']:.2f}s]"
476
477         # A: Reject segments that start beyond the video
478         if video_duration > 0 and c["start"] >= video_duration:
479             rejected.append(
480                 (
481                     label,
482                     f"start_time ({c['start']:.2f}s) is beyond video duration
({video_duration:.1f}s)",
483                 )
484             )
485             continue
486
487         # B: Clamp end_time to video duration
488         if video_duration > 0 and c["end"] > video_duration:
489             logger.info(
490                 f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping."
491             )
492             c["end"] = video_duration
493
494             dur = c["end"] - c["start"]
495
496         # C: Reject segments shorter than min_segment_duration
497         if dur < min_segment_duration:
498             rejected.append(
499                 (
500                     label,
501                     f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)",
502                 )
503             )
504             continue
505
506         # D: Flag suspiciously long segments

```



```

507         if dur > max_segment_duration:
508             logger.warning(
509                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
510                 f"This may be a detection error. Keeping but flagging."
511             )
512
513         validated.append(c)
514
515     # E: Resolve overlapping segments (keep the longer one)
516     if len(validated) > 1:
517         # Sort by start time
518         validated.sort(key=lambda r: r["start"])
519         non_overlapping = [validated[0]]
520         for curr in validated[1:]:
521             prev = non_overlapping[-1]
522             # Check if current segment overlaps with the previous one
523             if curr["start"] < prev["end"]:
524                 prev_dur = prev["end"] - prev["start"]
525                 curr_dur = curr["end"] - curr["start"]
526                 kept, dropped = (
527                     (prev, curr) if prev_dur >= curr_dur else (curr, prev)
528                 )
529                 kept_label = f"Segment #{kept['idx'] + 1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
530                 dropped_label = f"Segment #{dropped['idx'] + 1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
531                 rejected.append(
532                     (
533                         dropped_label,
534                         f"overlaps with {kept_label} ? kept the longer segment",
535                     )
536                 )
537                 if kept == curr:
538                     non_overlapping[-1] = curr # Replace previous with current
539                 else:
540                     non_overlapping.append(curr)
541             validated = non_overlapping
542
543     # Log validation summary
544     if rejected:
545         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
546         logger.info(
547             f"{len(rejected)} play segments were REJECTED:\n"
548             + "\n".join(reject_lines)
549         )
550     logger.info(
551         f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini)."
```

```

567         )
568
569         return validated
570
571     def _populate_db(
572         self, video_id: str, validated: List[Dict[str, Any]], model_name: str
573     ) -> List[Dict[str, Any]]:
574         """9. Populate SQLite DB with validated play segments; returns segments_data."""
575         segments_data = []
576         for c in validated:
577             start = c["start"]
578             end = c["end"]
579             segment = c["raw"]
580             dur = end - start
581
582             # Extract shot_count from Gemini or compute estimate if not present
583             shot_count = segment.get("shuttle_exchange_count")
584             if shot_count is None:
585                 shot_count = max(
586                     3, int(dur / 0.8)
587                 ) # 1 shot every 0.8 seconds as fallback estimate
588             else:
589                 try:
590                     shot_count = int(shot_count)
591                 except Exception:
592                     shot_count = max(3, int(dur / 0.8))
593
594             # Only store metrics we actually know ? no fabricated data
595             metadata = {
596                 "shot_count": shot_count,
597                 "shot_count_confidence": segment.get(
598                     "shot_count_confidence", "Unknown"
599                 ),
600                 "detector": model_name,
601             }
602
603             segment_id = self.db.add_play_segment(
604                 video_id, start, end, None, None, metadata
605             )
606
607             if segment_id:
608                 segments_data.append(
609                     {
610                         "id": segment_id,
611                         "start_time": start,
612                         "end_time": end,
613                         "duration": dur,
614                         "metadata": metadata,
615                     }
616                 )
617
618         return segments_data
619
620     def process_video(
621         self,
622         video_id: str,
623         video_path: str,
624         player_pool: List[str] = None,
625         max_frames: int = None,
626     ) -> List[Dict[str, Any]]:
627         """
628         Processes a video to detect play segments using Google Gemini Multimodal Video
629         API,
630         populates SQLite, and indexes detailed events.
631         """

```

```

631         model_name = self.config.ai_model
632
633         # Resolve sport profile + provider. An unrunnable config (unknown sport /
missing key /
634         # unknown provider) comes back as a Failure ? propagate it so the job is marked
FAILED
635         # with a reason, not a silent 0-rally success (Finding B).
636         resolved = self._resolve_provider(model_name)
637         if isinstance(resolved, Failure):
638             return resolved
639         sport_profile, provider = resolved
640
641         # Optionally trim the video first (debug feature); cleanup removes any temp
file.
642         video_to_upload, cleanup_temp_files = self._maybe_trim_video(
643             video_id, video_path
644         )
645
646         # 1. Determine actual video duration
647         video_duration = get_video_duration(video_to_upload)
648         if video_duration > 0:
649             logger.info(f"Detected video duration: {video_duration:.1f}s")
650         else:
651             logger.warning(
652                 "Could not detect video duration. Prompt will not include duration
context."
653             )
654
655         # 2. Decide whether to use chunked processing
656         idx_cfg = self.config.indexing
657         chunk_duration = idx_cfg.chunk_duration
658         chunk_overlap = idx_cfg.chunk_overlap
659         use_chunking = video_duration > 0 and video_duration > chunk_duration
660         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
661         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
662         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
663         ai_scale_width = int(idx_cfg.ai_chunk_scale_width or 0)
664
665         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
666         # video keeps its own dir, so cleanup here never removes another run's chunks.
667         chunks_dir = _chunk_workdir(video_id, output_base(self.config))
668
669         if use_chunking:
670             ai_segments = self._process_chunked(
671                 video_id,
672                 video_to_upload,
673                 sport_profile,
674                 provider,
675                 video_duration,
676                 chunk_duration,
677                 chunk_overlap,
678                 ai_scale_width,
679                 chunks_dir,
680                 cleanup_temp_files,
681             )
682         else:
683             ai_segments = self._process_single(
684                 video_id,
685                 video_to_upload,
686                 sport_profile,
687                 provider,
688                 video_duration,
689                 cleanup_temp_files,
690             )

```

```

691
692         if not ai_segments:
693             return []
694
695         # Populate SQLite DB with parsed play segments
696         validated = self._validate_and_dedupe(ai_segments, video_duration)
697
698         # 9. Populate SQLite DB with validated play segments
699         segments_data = self._populate_db(video_id, validated, model_name)
700
701         return segments_data
702
703
704     # Register the segmenter
705     segmenter_registry.register(GeminiPlaySegmenter)
706

```

20. assistant (2026-07-06T17:27:41.357Z)

Now I have a very clear picture. The key finding is emerging: the gemini segmenter treats ****missing API key / unknown sport / unknown provider**** as a ``Failure``, but the `trajectory_hybrid` fix did NOT ? it left those same paths returning bare ``[]``. This is the twin-parity inconsistency. Let me verify the remaining details: check ``cached_process_result`` continuation (does it require `status="success"`), and the `motion.py` precedent to confirm the established convention. Let me also confirm how ``_ingest_remote_segments`` behaves and the tail of orchestration.

21. user (2026-07-06T17:27:43.364Z)

```

349             return None
350         logger.info(
351             "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352             n,
353             video_id,
354             outputs_uri,
355         )
356         return _cached_process_payload(video_id, [], segments, requested_compute)
357     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358         logger.warning(
359             "[API] submit-time cache check failed for %s ? dispatching normally",
360             video_id,
361             exc_info=True,
362         )
363         return None
364
365
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372
373
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375
376
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports

```

22. user (2026-07-06T17:27:44.717Z)

```

6-import numpy as np
7-
8-from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9:from backend.results import Failure, Result, Success # standardized contract
10-
11-logger = logging.getLogger(__name__)
12-
13--
14-
36-        cap = cv2.VideoCapture(video_path)
37-        if not cap.isOpened():
38-            logger.error(f"Segmenter could not open video: {video_path}")
39-            return Failure(
40-                message=f"Could not open video: {video_path}", is_recoverable=False
41-            )
42-

```

23. assistant (2026-07-06T17:27:49.877Z)

This confirms the established convention across the codebase. Let me look at the full motion.py to see exactly which conditions it treats as Failure vs bare [] ? this establishes the precedent for whether "missing API key / bad config" should be Failure.

24. user (2026-07-06T17:27:50.507Z)

```

1    import logging
2    import time
3    from typing import Any, Dict, List, Optional, Tuple
4
5    import cv2
6    import numpy as np
7
8    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9    from backend.results import Failure, Result, Success # standardized contract
10
11    logger = logging.getLogger(__name__)
12
13
14    class MotionPlaySegmenter(BasePlaySegmenter):
15        @property
16        def name(self) -> str:
17            return "motion"
18
19        @property
20        def description(self) -> str:
21            return "Traditional CV pixel-motion heuristics tracking frame differences."
22
23        def process_video(
24            self,
25            video_id: str,
26            video_path: str,
27            player_pool: Optional[List[str]] = None,
28            max_frames: Optional[int] = None,
29        ) -> "Result[List[Dict[str, Any]]]":
30            """
31            Processes a video to detect play segments using motion heuristics and
32            populates SQLite with detected segments. The ``player_pool`` arg is
33            accepted for ABC signature compatibility but no longer used (the motion
34            segmenter has no ground truth for server/receiver ? see _populate_db).
35            """
36            cap = cv2.VideoCapture(video_path)
37            if not cap.isOpened():
38                logger.error(f"Segmenter could not open video: {video_path}")
39                return Failure(
40                    message=f"Could not open video: {video_path}", is_recoverable=False
41                )

```

```

42         final_segments = self._detect_motion_segments(cap, max_frames)
43
44         segments_data = self._populate_db(video_id, final_segments)
45
46         return Success(value=segments_data)
47
48     def _detect_motion_segments(
49         self, cap: Any, max_frames: Optional[int] = None
50     ) -> List[Tuple[float, float]]:
51         """Run the pixel-motion heuristic over ``cap`` and return the detected
52         (start, end) play segments.
53
54         Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
55         thresholds it into raw segments, then merges dead-time gaps and filters
56         out segments shorter than ``min_segment_duration``. ``cap`` is released
57         before returning. This is the detection half of ``process_video`` ? it
58         performs no DB writes and no fabrication.
59
60         ``max_frames`` limits the number of analysed frames (Optional; for debugging).
61         An explicit ``np.ones((3,3))`` kernel is used for ``cv2.dilate`` rather than
62         ``None`` so the typed overload resolves correctly.
63         """
64
65         fps = cap.get(cv2.CAP_PROP_FPS)
66         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
67         duration = total_frames / fps if fps > 0 else 0
68
69         # Sub-sample settings (5 FPS for analysis)
70         analysis_fps = 5.0
71         frame_interval = max(1, int(fps / analysis_fps))
72
73         idx_cfg = self.config.indexing
74         motion_threshold = idx_cfg.motion_threshold
75         min_segment_duration = idx_cfg.min_segment_duration
76         dead_time_merge_gap = idx_cfg.dead_time_merge_gap
77
78         prev_gray = None
79         motion_signals = []
80         timestamps = []
81
82         frame_idx = 0
83         processed_count = 0
84         t_start = time.time()
85         # Emit a progress line roughly every 5% of the video so long runs are
86         # observable instead of silent (a full 1080p match is ~tens of minutes).
87         progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
88         next_progress = progress_step
89         logger.info(
90             f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
91             f"(~{duration / 60:.1f} min), analyzing every {frame_interval}th frame"
92         )
93         while True:
94             # Skip decoding for intermediate frames using cap.grab() for high
95             performance
96                 for _ in range(frame_interval - 1):
97                     if not cap.grab():
98                         break
99                     frame_idx += 1
100
101                 ret, frame = cap.read()
102                 if not ret:
103                     break
104
105                 # Process frame
106                 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```

106         # Apply Gaussian Blur to smooth noise
107         gray = cv2.GaussianBlur(gray, (21, 21), 0)
108
109         t = frame_idx / fps
110         timestamps.append(t)
111
112         if prev_gray is not None:
113             # Frame absolute difference (highly optimized motion tracking)
114             diff = cv2.absdiff(prev_gray, gray)
115             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
116
117             # Dilate to fill gaps
118             kernel = np.ones((3, 3), np.uint8)
119             thresh = cv2.dilate(thresh, kernel, iterations=2)
120
121             # Motion ratio
122             motion_ratio = cv2.countNonZero(thresh) / (
123                 thresh.shape[0] * thresh.shape[1]
124             )
125             motion_signals.append(motion_ratio)
126         else:
127             motion_signals.append(0.0)
128
129         prev_gray = gray
130         frame_idx += 1
131         processed_count += 1
132
133         if progress_step and frame_idx >= next_progress:
134             pct = 100.0 * frame_idx / total_frames
135             elapsed = time.time() - t_start
136             eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
137             logger.info(
138                 f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames, "
139                 f"{processed_count} analyzed, {elapsed:.0f}s elapsed, ~{eta:.0f}s
left"
140             )
141             next_progress += progress_step
142
143         if max_frames is not None and processed_count >= max_frames:
144             logger.debug(
145                 f"Reached max_frames limit of {max_frames}. Halting indexer frame
parsing."
146             )
147             break
148
149         if frame_idx >= total_frames:
150             break
151
152     cap.release()
153
154     # Smooth signal using a moving average
155     window_size = 5
156     smoothed = []
157     for i in range(len(motion_signals)):
158         start = max(0, i - window_size // 2)
159         end = min(len(motion_signals), i + window_size // 2 + 1)
160         smoothed.append(sum(motion_signals[start:end]) / (end - start))
161
162     # Detect active play boundaries
163     in_segment = False
164     segment_start: Optional[float] = None
165     raw_segments: List[Tuple[float, float]] = []
166
167     for i, t in enumerate(timestamps):
168         signal = smoothed[i]

```

```

169         if signal > motion_threshold:
170             if not in_segment:
171                 in_segment = True
172                 segment_start = t
173             else:
174                 if in_segment:
175                     in_segment = False
176                     assert segment_start is not None # set when in_segment became True
177                     raw_segments.append((segment_start, t))
178
179     # Handle video ending while in active segment
180     if in_segment:
181         assert segment_start is not None # set when in_segment became True
182         raw_segments.append((segment_start, timestamps[-1]))
183
184     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
185     merged_segments: List[Tuple[float, float]] = []
186     if raw_segments:
187         curr_start, curr_end = raw_segments[0]
188         for next_start, next_end in raw_segments[1:]:
189             if next_start - curr_end < dead_time_merge_gap:
190                 # Merge
191                 curr_end = next_end
192             else:
193                 merged_segments.append((curr_start, curr_end))
194                 curr_start, curr_end = next_start, next_end
195         merged_segments.append((curr_start, curr_end))
196
197     # Post-Processing: 2. Filter segments shorter than min_segment_duration
198     final_segments: List[Tuple[float, float]] = [
199         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
200     ]
201
202     return final_segments
203
204 def _populate_db(
205     self, video_id: str, final_segments: List[Tuple[float, float]]
206 ) -> List[Dict[str, Any]]:
207     """Persist ``final_segments`` to the DB with honest metadata.
208
209     The motion segmenter detects play segments via pixel-diff heuristics but
210     has NO ground truth for who served, shot counts, shuttle speed, or event
211     breakdown ? those were previously fabricated with the stdlib ``random``
212     module (the legacy demo baseline). They are now persisted as ``None`` /
213     unknown markers so consumers can distinguish "not modeled" from a real
214     value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
215     """
216     segments_data = []
217     for start, end in final_segments:
218         dur = end - start
219         metadata = {
220             "shot_count": None,
221             "intensity": "high" if dur > 12 else "medium",
222             "avg_speed_kmh": None,
223         }
224         segment_id = self.db.add_play_segment(
225             video_id, start, end, None, None, metadata
226         )
227         if segment_id:
228             segments_data.append(
229                 {
230                     "id": segment_id,
231                     "start_time": start,
232                     "end_time": end,
233                     "duration": dur,

```



```

234             "server": None,
235             "receiver": None,
236             "metadata": metadata,
237         }
238     )
239     return segments_data
240
241
242     # Register the segmenter
243     segmenter_registry.register(MotionPlaySegmenter)
244

```

25. assistant (2026-07-06T17:27:54.404Z)

Now I have the complete picture. Let me verify one more thing about the wf294 incident specifically: was the WASB native runner's "produced no trajectory" the exact path, or could the timeout manifest as an empty CSV that parses to 0 points (which would flow past the `if not csv\_path` guard into `if not all\_candidate\_windows: return []`)? Let me check the native WASB runner's run_predict to see what it returns on timeout.

26. user (2026-07-06T17:27:56.203Z)

Found 5 files
 backend\pipeline\detectors\native_wasb_runner.py
 backend\pipeline\detectors\base.py
 backend\pipeline\detectors\tracknet_runner.py
 backend\pipeline\detectors\wasb_runner.py
 backend\pipeline\detectors\stub_runner.py

27. user (2026-07-06T17:28:00.083Z)

```

78-     sport: str = "badminton"
79-     device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80-     scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81-     timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82-     keep_frames: bool = (
83-         False # retain the decoded frame cache after success (debug only)
84-     )
85-
86-     --
110-         or env("RALLY_SCRATCH")
111-         or env("TMPDIR")
112-         or "",
113-         timeout_sec=w.timeout_sec,
114-         keep_frames=w.keep_frames,
115-         stream_video=(True if sv is None else bool(sv)),
116-         batch_size=int(w.batch_size or 0),
117-     )
118-
119-     --
137-         cwd=cwd,
138-         capture_output=True,
139-         text=True,
140-         timeout=(self.cfg.timeout_sec or None),
141-     )
142-
143-     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-cpu)
144-     -----
145-     def _probe_cuda(self) -> bool:
146-         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess). Any
147-         failure
148-         (missing python / timeout / import error) is treated as 'no CUDA'."""
149-         try:
150-             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
151-         except (FileNotFoundError, subprocess.TimeoutExpired):
152-             return False
153-         return res.returncode == 0 and "cuda True" in (res.stdout or "")

```

```

152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-
368-
369-
370-
371-
372-
373-
374-
375-
376-
377-
378-
379-
380-
381-
382-
383-
384-
385-
386-
387-
388-
389-
390-
391-
392-
393-
394-
395-
396-
397-
398-
399-
400-
401-
402-
403-
404-
405-
406-
407-
408-
409-
410-
411-
412-
413-
414-
415-
416-
417-
418-
419-
420-
421-
422-
423-
424-
425-
426-
427-
428-
429-
430-
431-
432-
433-
434-
435-
436-
437-
438-
439-
440-
441-
442-
443-
444-
445-
446-
447-
448-
449-
450-
451-
452-
453-
454-
455-
456-
457-
458-
459-
460-
461-
462-
463-
464-
465-
466-
467-
468-
469-
470-
471-
472-
473-
474-
475-
476-
477-
478-
479-
480-
481-
482-
483-
484-
485-
486-
487-
488-
489-
490-
491-
492-
493-
494-
495-
496-
497-
498-
499-
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518-
519-
520-
521-
522-
523-
524-
525-
526-
527-
528-
529-
530-
531-
532-
533-
534-
535-
536-
537-
538-
539-
540-
541-
542-
543-
544-
545-
546-
547-
548-
549-
550-
551-
552-
553-
554-
555-
556-
557-
558-
559-
560-
561-
562-
563-
564-
565-
566-
567-
568-
569-
570-
571-
572-
573-
574-
575-
576-
577-
578-
579-
580-
581-
582-
583-
584-
585-
586-
587-
588-
589-
590-
591-
592-
593-
594-
595-
596-
597-
598-
599-
600-
601-
602-
603-
604-
605-
606-
607-
608-
609-
610-
611-
612-
613-
614-
615-
616-
617-
618-
619-
620-
621-
622-
623-
624-
625-
626-
627-
628-
629-
630-
631-
632-
633-
634-
635-
636-
637-
638-
639-
640-
641-
642-
643-
644-
645-
646-
647-
648-
649-
650-
651-
652-
653-
654-
655-
656-
657-
658-
659-
660-
661-
662-
663-
664-
665-
666-
667-
668-
669-
670-
671-
672-
673-
674-
675-
676-
677-
678-
679-
680-
681-
682-
683-
684-
685-
686-
687-
688-
689-
690-
691-
692-
693-
694-
695-
696-
697-
698-
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-
1000-

```

28. assistant (2026-07-06T17:28:07.066Z)

This confirms the exact wf294 path: `native\_wasb\_runner.run\_predict` returns `None` on `TimeoutExpired` (line 306-310), which the fix now catches at `trajectory\_hybrid.py:290` `if not csv\_path: ? returns `Failure`. That path is correctly fixed.

Now I have everything to assess completeness. Let me quickly confirm that the wasb_hybrid and tracknet_hybrid are thin subclasses (so the fix applies to them via the shared base) and check what tests exist for yolo_hybrid failure behavior.

29. user (2026-07-06T17:28:08.635Z)

```
1      """WASB-based hybrid play segmenter.
2
3      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15
16     from typing import Any, Dict, Tuple
17
18     from backend.config.models import IndexingConfig
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
21     from backend.pipeline.segmenters.base import segmenter_registry
22     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
23
24
25     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
26         family = "wasb"
27         display_label = "WASB"
28
29         @property
30         def name(self) -> str:
31             return "wasb_hybrid"
32
33         @property
34         def description(self) -> str:
35             return (
36                 "Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
37                 badminton "
38                 "weights) for precise candidate rallies + AI provider for semantic
39                 boundaries."
40             )
41
42         def _build_runner(
43             self, idx_cfg: IndexingConfig
44         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
45             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
46             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
47
48         def _build_native_runner(
49             self, idx_cfg: IndexingConfig
50         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
51             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
52             # seam (imported only when detector_impl='native' is actually selected).
53             stub
54             from backend.pipeline.detectors.native_wasb_runner import (
55                 NativeWasbConfig,
56                 NativeWasbRunner,
57             )
58
59             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
```

```

57         return NativeWasbRunner(cfg), {
58             "weights_path": cfg.weights_path,
59             "detector_impl": "native",
60             "device": cfg.device,
61         }
62
63
64     segmenter_registry.register(WasbHybridSegmenter)
65

```

30. user (2026-07-06T17:28:10.119Z)

```

C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer\backend\config\models.py-179-
backend\config\models.py-180-class IndexingConfig(BaseModel):
backend\config\models.py:181:     default_segmenter: str = "yolo_hybrid"
backend\config\models.py-182-     use_gpu: bool = True
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer\backend\config\models.py-183-
# Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
--
backend\config\models.py-273-     fusion: FusionConfig = Field(default_factory=FusionConfig)
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer\backend\config\models.py-274-
backend\config\models.py:275:     @field_validator("default_segmenter")
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer\backend\config\models.py-276-
@classmethod
backend\config\models.py-277-     def segmenter_must_be_registered(cls, v: str) -> str:
--
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\config\loader.py-254-# local storage and the compute_target, so this stays the
handful of knobs a fresh, non-technical
backend\config\loader.py-255-# user gets ? a torch-free, offline, zero-setup experience:
backend\config\loader.py:256:#     * default_segmenter "motion" ? pure-OpenCV, no torch / GPU /
weights / API key (LIGHT tier);
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\config\loader.py-257-#     * profile "truly-local" ? no Gemini handoff,
local storage ? never surprise-bills on
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\config\loader.py-258-# Gemini nor returns 0
rallies on WASB-without-a-key (08 intent);
--
backend\config\loader.py-262-     "sport": "badminton",
backend\config\loader.py-263-     "deployment": {"profile": "truly-local"},
backend\config\loader.py:264:     "indexing": {"default_segmenter": "motion", "use_gpu": False},
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\config\loader.py-265-}
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer\backend\config\loader.py-266-

```

31. assistant (2026-07-06T17:28:27.462Z)

I now have the complete picture. Let me finalize my analysis of the completeness gaps.

The wf294 incident was specifically: WASB detector timed out (1800s) ? `run\_predict` returned `None` ? `if not csv\_path` ? bare `[]` ? status="success". The fix correctly addresses this exact path and the healthcheck path.

Now examining the **remaining bare-`[]` returns in `trajectory\_hybrid.process\_video`** and classifying them:

- Line 194 ? unknown sport (KeyError)**:** This is a misconfiguration, not "ran, found nothing." The gemini/motion precedent treats this as `Failure`. Left as bare `[]` ? false "success".
- Line 219 ? missing API key**:** This is the exact scenario the worker's own diagnostic warning (line 340-352) explicitly calls out as a cause of a false 0-segment "success" ("a missing \${AI_PROVIDER}_API_KEY... the segmenter bails BEFORE detection"). The gemini segmenter's `\_resolve\_provider` treats this as `Failure` (its docstring literally cites "Finding B ? the loud-failures bar"). Left as bare `[]` here ? the SAME silent-failure class as wf294.

3. **Line 226** ? unknown provider (KeyError): Same as above; gemini treats as ``Failure``.
4. **Line 231** ? invalid video_duration <= 0: Detector/input failure, not an empty match. motion.py treats "could not open video" as ``Failure``.
5. **Line 244** ? ``_resolve_runner`` NotImplementedError (native on a non-WASB family): Config/env failure, not an empty match.
6. **Line 371** ? ``if not all_candidate_windows: return []``: This is legitimately "ran, found nothing" (trajectory was produced+parsed). Correctly left as ``[]``.
7. **Phase-3 all-handoffs-fail** ? empty ``ai_segments``: Provider errors on every window. Notably, the per-window helper logs a warning on ``metrics["error"]`` but the aggregate still returns ``[]`` with no distinction. This is a partial-failure silent gap, but it is pre-existing and shared with yolo/gemini (which also don't fail the run on all-windows-errored) ? out of the wf294 minimal scope and arguably a larger design question. I'll note it as lower priority.

The strongest, most concrete completeness gaps are **the missing-API-key path (line 219)** and **the sibling yolo_hybrid segmenter**.

Let me verify the yolo_hybrid concern precisely: it is the **default segmenter** (``default_segmenter = "yolo_hybrid"``) and is worker-dispatchable, and its ``process_video`` returns bare ``[]`` for missing API key / unknown sport / unknown provider (via ``_resolve_provider`` returning ``None`` ? line 470-471) with NO ``Failure`` at all. So a yolo_hybrid run with a missing API key reproduces the exact wf294 silent-failure shape and was left completely untouched by the fix.

Now let me submit the findings.

32. user (2026-07-06T17:29:03.294Z)

Structured output provided successfully